

Factory Method Design Pattern

Senior/Staff Interview Handbook — Saudi Arabia, Dubai/UAE, Malaysia, India
Tier-2, India Tier-1/60LPA+ (Bilingual English + Hindi)

Interview Prep Handbook · Updated August 2026

Factory Method Design Pattern

Fast Track (Parts 1–6) → Deep Dive (Parts 7–21) → Appendix (Part 22). Companion code: `Factory.php` — all runnable examples referenced by name live there, not inlined here (code file is English-only).

फ़ास्ट ट्रैक (भाग 1–6) → डीप डाइव (भाग 7–21) → परिशिष्ट (भाग 22)। साथी कोड फ़ाइल: `Factory.php` — नाम से रेफ़र किए गए सारे रनेबल उदाहरण वहीं हैं, यहाँ इनलाइन नहीं (कोड फ़ाइल सिर्फ़ अंग्रेज़ी में है)।

⚡ FAST TRACK

Part 1 — 60-Second Recall Card

One-liner	Factory Method defines a method for creating an object, but lets the class that implements it decide which concrete class actually gets instantiated.
GoF category	Creational
Core mechanism	An abstract <code>Creator</code> declares a <code>factoryMethod()</code> that returns a <code>Product</code> interface/abstract type. Each <code>ConcreteCreator</code> overrides <code>factoryMethod()</code> to return a specific <code>ConcreteProduct</code> . The rest of the <code>Creator</code> 's logic — its non-factory methods — is written entirely against the <code>Product</code> interface, so it never needs to know which concrete class it's actually working with.

Trigger phrase	"We don't know the exact class of object we need until runtime"; "adding a new type shouldn't require touching existing client code"; "object-creation logic is duplicated across the codebase and varies by type."
Anti-trigger	Only one concrete product type will ever exist (a plain constructor is enough); you need a <i>family</i> of related objects created together, not one object (→ Abstract Factory); the variation is in <i>how</i> an object is assembled step by step, not <i>which class</i> it is (→ Builder); you just need a single centralized switch-statement with no subclassing/extensibility requirement (→ a plain "Simple Factory," which is a useful pragmatic idiom but not this GoF pattern).
Closest confused patterns	Abstract Factory — a <i>family</i> of related factory methods bundled behind one interface, vs. Factory Method's single creation method. Simple Factory / static factory — not a GoF pattern at all; a static method with an <code>if / switch</code> picking a class, with no subclassing or polymorphic override involved — extremely common in real code and extremely commonly mislabeled as "the Factory pattern." Builder — multi-step, optional-parameter-heavy construction of one complex object, vs. Factory Method's single-step decision of <i>which</i> class to instantiate. Prototype — clones an existing configured instance rather than deciding among classes to instantiate fresh.
Memory hook	A pizza-chain franchise: every franchise (<code>ConcreteCreator</code>) follows the identical "take order → bake → box → hand over" process (the <code>Creator</code> 's ordinary methods), but each franchise's <code>makePizza()</code> (the factory method) returns a different <code>Pizza</code> depending on the regional menu — Chicago deep-dish here, Neapolitan there — without the shared ordering process ever needing to know which one it got.

हिंदी अनुवाद / Hindi Translation:

एक-पंक्ति सार	Factory Method एक ऑब्जेक्ट बनाने के लिए एक मेथड डिफ़ाइन करता है, लेकिन यह तय करना उस क्लास पर छोड़ देता है जो उसे इम्प्लीमेंट करती है कि असल में कौन-सी कॉन्क्रीट क्लास इंस्टैंशिएट होगी।
GoF श्रेणी	क्रिएशनल (Creational)
मुख्य तंत्र	एक ऐब्सट्रेक्ट <code>Creator</code> , एक <code>factoryMethod()</code> डिक्लेयर करता है जो <code>Product</code> इंटरफ़ेस/ऐब्सट्रेक्ट टाइप लौटाता है। हर <code>ConcreteCreator</code> , <code>factoryMethod()</code> को ओवरराइड करके एक खास <code>ConcreteProduct</code> लौटाता है। <code>Creator</code> का बाक़ी लॉजिक — उसके गैर-factory मेथड्स — पूरी तरह <code>Product</code> इंटरफ़ेस के खिलाफ़ लिखा जाता है, इसलिए उसे कभी यह जानने की ज़रूरत नहीं पड़ती कि वह असल में किस कॉन्क्रीट क्लास के साथ काम कर रहा है।
ट्रिगर वाक्यांश	"हमें ठीक-ठीक नहीं पता कि रनटाइम तक हमें किस क्लास का ऑब्जेक्ट चाहिए"; "नई टाइप जोड़ने के लिए मौजूदा क्लाइट कोड को छूना नहीं पड़ना चाहिए"; "ऑब्जेक्ट-क्रिएशन लॉजिक कोडबेस में दोहराया गया है और टाइप के हिसाब से बदलता है।"
एंटी-ट्रिगर	सिर्फ़ एक ही कॉन्क्रीट प्रोडक्ट टाइप कभी अस्तित्व में रहेगा (एक साधारण कंस्ट्रक्टर काफ़ी है); आपको एक साथ बनाए गए संबंधित ऑब्जेक्ट्स का एक परिवार चाहिए, एक ऑब्जेक्ट नहीं (→ Abstract Factory); बदलाव इसमें है कि ऑब्जेक्ट कैसे चरण-दर-चरण असेंबल होता है, कौन-सी क्लास नहीं (→ Builder); आपको बस एक केंद्रीकृत switch-स्टेटमेंट चाहिए, बिना किसी सबक्लासिंग/एक्सटेंसिबिलिटी ज़रूरत के (→ एक सादा "Simple Factory", जो एक उपयोगी व्यावहारिक तरीक़ा है लेकिन यह GoF पैटर्न नहीं है)।
सबसे मिलते-जुलते	Abstract Factory — एक इंटरफ़ेस के पीछे बंधी हुई संबंधित factory मेथड्स का एक परिवार, बनाम Factory Method की एक अकेली क्रिएशन मेथड। Simple Factory / स्टैटिक फ़ैक्टरी — बिल्कुल भी GoF पैटर्न नहीं; एक क्लास चुनने वाली <code>if / switch</code> वाली एक स्टैटिक मेथड, कोई सबक्लासिंग या पॉलीमॉर्फिक ओवरराइड शामिल नहीं — असली कोड में बेहद आम, और बेहद आम तौर पर "The Factory pattern" ग़लत नाम से जानी जाती है। Builder — एक जटिल ऑब्जेक्ट का बहु-चरणीय, वैकल्पिक-

भ्रामक पैटर्न्स	पैरामीटर-भारी निर्माण, बनाम Factory Method का यह एक-चरणीय फ़ैसला कि कौन-सी क्लास इंस्टैंशिएट करनी है। Prototype — नए सिरे से इंस्टैंशिएट करने के लिए क्लासेज़ में से चुनने के बजाय एक मौजूदा कॉन्फ़िगर्ड इंस्टेंस को क्लोन करता है।
याद रखने की तरकीब	एक पिज़्ज़ा-चेन फ्रैंचाइज़ी: हर फ्रैंचाइज़ी (ConcreteCreator) बिल्कुल एक जैसी "ऑर्डर लो → बेक करो → बॉक्स करो → सॉपो" प्रक्रिया फ़ॉलो करती है (Creator के सामान्य मेथड्स), लेकिन हर फ्रैंचाइज़ी का makePizza() (factory मेथड) क्षेत्रीय मेन्यू के हिसाब से अलग Pizza लौटाता है — यहाँ शिकागो डीप-डिश, वहाँ नियापोलिटन — बिना साझा ऑर्डरिंग प्रक्रिया को कभी यह जानने की ज़रूरत पड़े कि उसे कौन-सा मिला।

Part 2 — Market Calibration

Sourced directly from `design-patterns-frequency-guide-expanded.md`. Two honesty notes up front, because they shape how to read every number below: (1) the guide tracks this pattern as a single combined "Factory / Abstract Factory" bucket — it does not separately measure Factory Method vs. Abstract Factory frequency, so the market data below reflects "some flavor of Factory" being asked, not confirmation that it was specifically Factory Method every time; (2) unlike Singleton (which had a specific "double-checked locking" implementation detail tied to it), the guide records no company-specific implementation detail for Factory anywhere — only pattern-name-list mentions — so this section is calibrated on frequency, not depth-of-probing evidence.

सीधे `design-patterns-frequency-guide-expanded.md` से लिया गया। शुरुआत में दो ईमानदार टिप्पणियाँ, क्योंकि ये तय करती हैं कि नीचे हर आँकड़े को कैसे पढ़ा जाए: (1) गाइड इस पैटर्न को एक ही संयुक्त "Factory / Abstract Factory" बकेट के तौर पर ट्रैक करती है — यह Factory Method बनाम Abstract Factory की फ्रीक्वेंसी अलग-अलग नहीं मापती, इसलिए नीचे का बाज़ार डेटा "किसी न किसी तरह की Factory" पूछे जाने को दर्शाता है, यह पुष्टि नहीं कि हर बार खास तौर पर Factory Method ही थी; (2) Singleton के उलट (जिसके साथ एक खास "double-checked locking" इम्प्लीमेंटेशन डिटेल जुड़ी थी), गाइड कहीं भी Factory के लिए कोई कंपनी-विशिष्ट इम्प्लीमेंटेशन डिटेल दर्ज नहीं करती — सिर्फ़ पैटर्न-नाम-सूची वाले ज़िक्र — इसलिए यह सेक्शन फ्रीक्वेंसी पर कैलिब्रेट है, गहराई-से-जाँच के सबूत पर नहीं।

Factory/Abstract Factory ranks **#2 overall** on the master frequency table (just behind Strategy), labeled **Very High**, with real-world anchors: Parking Lot, Vehicle systems, Notification channels.

मास्टर फ्रीक्वेंसी टेबल पर Factory/Abstract Factory **कुल मिलाकर #2** पर है (Strategy से बस पीछे), जिसे **Very High** का दर्जा दिया गया है, असली-दुनिया के एंकर्स के साथ: Parking Lot, Vehicle systems, Notification channels।

Market	Factory's standing	Evidence	What that means for prep
India Tier-2	Strongest market — explicitly top-4, most heavily evidenced.	Market summary: "Strategy is the clear #1... Observer, Factory, Singleton round out the top 4." Named at Razorpay and Postman (both confirmed, Postman explicitly noted for justifying pattern choices out loud), plus ShareChat, Delhivery,	Very likely to come up in a machine-coding round for anything with multiple similar-but-different types to construct (payment methods, notification channels, vehicle types)

Market	Factory's standing	Evidence	What that means for prep
		Dream11, Rapido, Paytm, Ola, Infosys, Cognizant (confirmed/ inferred mix), and inferred at LTIMindtree, TCS Digital, Capgemini, Lenskart, upGrad.	— high confidence this is a "name it and justify it" expectation here.
India Tier-1 / 60LPA+	Strong secondary pattern , grouped with Singleton/Builder just behind Strategy.	Market summary: "Strategy again leads..., with Singleton, Factory/ Builder, and Observer close behind." Named at Directi/Media.net (confirmed, explicit, alongside seven other named patterns) and Mastercard India (confirmed-light — though the specific "Burger-Shop" example the guide records there reads as more Builder-attributed than Factory-attributed), and inferred at Grab (India-facing).	Expect it as one of several patterns a bar-raiser wants named in a larger design — supporting player, not usually the sole focus, with real scrutiny on whether you can defend <i>why</i> Factory over a simpler <code>if / switch</code> .
Dubai/ UAE	Named as a co-leader in the market summary, but the guide's own underlying evidence is thin and shows an internal inconsistency worth knowing about.	Market summary states "Strategy and Factory lead" — yet the master table's own citation for this ("UAE (Property Finder)") doesn't hold up under closer reading: Property Finder's individual company row records an explicit LLD round with SAGA and microservice orchestration, but doesn't itself name Factory. Only company row actually naming it: Noon.com (inferred only — "Strategy/ Factory/Observer (inferred)").	Treat the "Factory leads in UAE" framing with real caution — it's asserted at the summary level but not well-anchored in the guide's own company-level data. Prepare Factory at solid baseline depth for UAE, not at a level implying it's unusually dominant there specifically.
Malaysia	Solid supporting pattern, not called out in the market's own top-line summary.	Malaysia's "most-asked pattern" line names Singleton and Strategy, not Factory — but company rows still show real presence: Shopee (confirmed-regional, Hard), Carsome (confirmed, Medium), and inferred at Grab (MY hub), Boost/Axiata, IBM Malaysia.	Solid, expected supporting pattern — don't over-invest specifically for Malaysia beyond baseline, since the market's headline signal is Singleton, not Factory.
Saudi Arabia	Thin, mostly inferred.	Only confirmed row: Accenture (KSA centers) — global-confidence data, Factory flagged "most important" alongside Strategy/Observer/Singleton. Salla and Zid both show Factory only	Prepare at baseline depth; don't assume Saudi-specific emphasis beyond the general "Very High" global frequency — the country-specific

Market	Factory's standing	Evidence	What that means for prep
		as an inferred guess, not confirmed evidence.	evidence here is genuinely light.

हिंदी अनुवाद / Hindi Translation (टेबल का सार, बाज़ार दर हिसाब से):

India Tier-2 — सबसे मज़बूत बाज़ार, साफ़ तौर पर टॉप-4 में, सबसे ज़्यादा सबूतों के साथ। बाज़ार सार कहता है: "Strategy साफ़ तौर पर #1 है... Observer, Factory, Singleton टॉप-4 पूरा करते हैं।" Razorpay और Postman (दोनों पुष्ट) पर नामित, साथ ही ShareChat, Delhivery, Dream11, Rapido, Paytm, Ola, Infosys, Cognizant (पुष्ट/अनुमानित मिश्रण), और LTIMindtree, TCS Digital, Capgemini, Lenskart, upGrad पर अनुमानित। तैयारी के लिए मतलब: किसी भी ऐसी चीज़ के लिए मशीन-कोडिंग राउंड में बहुत आने की संभावना जिसमें कई मिलते-जुलते-मगर-अलग टाइप्स बनाने हों (पेमेंट मेथड्स, नोटिफिकेशन चैनल्स, वाहन टाइप्स) — यहाँ "नाम बताओ और सही ठहराओ" की अपेक्षा पर उच्च भरोसा।

India Tier-1 / 60LPA+ — Strategy के ठीक पीछे, Singleton/Builder के साथ समूहबद्ध एक मज़बूत द्वितीयक पैटर्न। Directi/Media.net (पुष्ट) और Mastercard India (हल्का-पुष्ट) पर नामित, Grab (भारत-केंद्रित) पर अनुमानित। तैयारी के लिए मतलब: एक बड़े डिज़ाइन में एक बार-रेज़र द्वारा नामित किए जाने वाले कई पैटर्न्स में से एक की उम्मीद रखें — सहायक भूमिका, आमतौर पर अकेला केंद्र-बिंदु नहीं, और यह असली जाँच होगी कि क्या आप यह बचाव कर सकते हैं कि एक सरल `if / switch` के बजाय Factory क्यों।

Dubai/UAE — बाज़ार सार में सह-नेता के तौर पर नामित, लेकिन गाइड का अपना ही अंतर्निहित सबूत पतला है और एक आंतरिक असंगति दिखाता है जो जानने लायक है। सार कहता है "Strategy और Factory आगे हैं" — लेकिन इसका मास्टर टेबल का अपना उद्धरण ("UAE (Property Finder)") क़रीब से पढ़ने पर टिकता नहीं। तैयारी के लिए मतलब: "UAE में Factory आगे है" वाली बात को सावधानी से लें — इसे मज़बूत बुनियादी गहराई पर तैयार करें, यह मान लिए बिना कि यह वहाँ असामान्य रूप से हावी है।

Malaysia — एक ठोस सहायक पैटर्न, बाज़ार के अपने हेडलाइन सार में शामिल नहीं। "सबसे ज़्यादा पूछा गया पैटर्न" वाली पंक्ति Singleton और Strategy को नाम देती है, Factory को नहीं — लेकिन कंपनी-पंक्तियाँ असली मौजूदगी दिखाती हैं: Shopee, Carsome (पुष्ट), और Grab (MY hub), Boost/Axiata, IBM Malaysia पर अनुमानित। तैयारी के लिए मतलब: ठोस, अपेक्षित सहायक पैटर्न — Malaysia के लिए बुनियादी स्तर से ज़्यादा अतिरिक्त निवेश न करें।

Saudi Arabia — पतला, ज़्यादातर अनुमानित। सिर्फ़ पुष्ट पंक्ति: Accenture (KSA केंद्र) — वैश्विक-भरोसे वाला डेटा। Salla और Zid दोनों में Factory सिर्फ़ एक अनुमान के तौर पर दिखता है, पुष्ट सबूत नहीं। तैयारी के लिए मतलब: बुनियादी गहराई पर तैयारी करें; सामान्य वैश्विक "Very High" फ़्रीक्वेंसी से आगे सज़ुदी-विशिष्ट ज़ोर न मानें।

Bottom line: Factory Method (bundled with Abstract Factory in this dataset) is the guide's #2 pattern overall and India Tier-2's strongest, best-evidenced supporting pattern. Treat the UAE "Factory leads" framing skeptically — it's a summary-level claim the guide's own company rows don't fully back up — and don't expect the kind of single, sharp implementation-detail follow-up (like Singleton's double-checked locking) that some other patterns get; the real test here is almost always "why Factory and not a plain conditional," not a specific mechanism deep-dive.

निचोड़: Factory Method (इस डेटासेट में Abstract Factory के साथ बंडल्ड) गाइड का कुल मिलाकर #2 पैटर्न है और India Tier-2 का सबसे मज़बूत, सबसे अच्छे-सबूत वाला सहायक पैटर्न है। UAE की "Factory आगे है" वाली बात को संदेह के साथ लें — यह सार-स्तर का दावा है जिसे गाइड की अपनी कंपनी-पंक्तियाँ पूरी तरह समर्थन नहीं देती — और किसी एक, तीखे इम्प्लीमेंटेशन-डिटेल फ़ॉलो-अप (जैसे Singleton का double-checked locking) की उम्मीद न रखें; यहाँ असली परीक्षा लगभग हमेशा "Factory ही क्यों, एक सादा conditional क्यों नहीं" होती है, किसी खास तंत्र की गहरी जाँच नहीं।

Part 3 — Recognition, Decision Tree & When NOT to Use

Requirement phrases that signal Factory Method: - "The exact class of object needed depends on configuration/input/environment, and isn't known until runtime." - "We need to support new types in the future without modifying the code that already uses this." - "This constructor call is duplicated across many places, with a type flag or conditional deciding which class to build each time." - "Different subclasses/modules should be able to create different variations of the same kind of thing."

Factory Method का संकेत देने वाले शब्द/वाक्यांश: - "आवश्यक ऑब्जेक्ट की ठीक-ठीक क्लास कॉन्फिगरेशन/इनपुट/एनवायरनमेंट पर निर्भर करती है, और रनटाइम तक पता नहीं होती।" - "भविष्य में हमें नई टाइप्स सपोर्ट करनी हैं, बिना उस कोड को बदले जो इसे पहले से इस्तेमाल कर रहा है।" - "यह कंस्ट्रक्टर कॉल कई जगहों पर दोहराई गई है, एक टाइप फ्लैग या शर्त हर बार तय करती है कि कौन-सी क्लास बनानी है।" - "अलग-अलग सबक्लासेज़/मॉड्यूल्स को एक ही तरह की चीज़ के अलग-अलग रूप बनाने में सक्षम होना चाहिए।"

Code smells that signal an existing Factory Method opportunity (or a broken/missing one): - A repeated `if ($type === 'x') { new X(); } elseif ($type === 'y') { new Y(); }` block, copy-pasted or nearly identical, across multiple files. - Adding a new supported type requires hunting down and editing every one of those conditional blocks — a direct Open/Closed Principle violation. - Client code holding a direct, hard-coded dependency on multiple concrete classes it only ever uses through one shared interface.

कोड स्मेल्स जो Factory Method के मौक़े (या टूटे/गायब एक) का संकेत देते हैं: - एक दोहराया हुआ `if ($type === 'x') { new X(); } elseif ($type === 'y') { new Y(); }` ब्लॉक, कई फ़ाइलों में कॉपी-पेस्ट या लगभग एक जैसा। - एक नई सपोर्टेड टाइप जोड़ने के लिए उन शर्तों वाले हर ब्लॉक को ढूँढ़ना और एडिट करना पड़ता है — सीधा Open/Closed Principle उल्लंघन। - क्लाइंट कोड कई कॉन्क्रीट क्लासेज़ पर सीधी, हार्ड-कोडेड निर्भरता रखता है, जबकि उसे सिर्फ़ एक साझा इंटरफ़ेस के ज़रिए इस्तेमाल करना चाहिए था।

Decision tree:

Does the concrete type genuinely vary – will more than one class ever need to be created here, now or in a foreseeable future?

- └ NO → Just use a constructor directly. Don't build factory machinery for a type that will never have a second variant.
- └ YES → Are you creating a whole FAMILY of related objects together (e.g., a button AND a checkbox that must match one visual theme), not just one object?
 - └ YES → Abstract Factory, not Factory Method.
 - └ NO, just one kind of object → Does building it involve many optional/combinable configuration steps, more than "which class" varies?
 - └ YES → Builder is probably the better fit.
 - └ NO → Do you need subclasses to be able to override and extend the creation logic polymorphically (real OOP extensibility need), or would a single static method with a match/switch genuinely be simpler and sufficient?
 - └ Need real polymorphic extensibility → Factory Method
 - └ A simple, centralized switch is enough → "Simple Factory" (pragmatic, not a GoF pattern, and that's fine to say explicitly in an interview)

निर्णय वृक्ष (डिसीज़न ट्री) — हिंदी सार: पहले पूछें: क्या कॉन्क्रीट टाइप सचमुच बदलती है — क्या यहाँ कभी एक से ज़्यादा क्लास बनानी पड़ेगी? अगर नहीं, तो सीधे कंस्ट्रक्टर इस्तेमाल करें। अगर हाँ, तो पूछें: क्या आप संबंधित ऑब्जेक्ट्स का एक पूरा परिवार एक साथ बना रहे हैं (जैसे एक बटन और एक चेकबॉक्स जिन्हें एक ही थीम से मेल खाना चाहिए)? अगर हाँ, तो यह Abstract Factory है, Factory Method नहीं। अगर सिर्फ़ एक तरह का ऑब्जेक्ट है, तो पूछें: क्या इसे बनाने में कई वैकल्पिक/संयोजनीय कॉन्फ़िगरेशन चरण शामिल हैं, "कौन-सी क्लास" से ज़्यादा? अगर हाँ, तो Builder ज़्यादा उपयुक्त है। अगर नहीं, तो अंतिम सवाल: क्या सबक्लासेज़ को क्रिएशन लॉजिक को पॉलीमॉर्फिक रूप से ओवरराइड और एक्सटेंड करने में सक्षम होना चाहिए (असली OOP एक्सटेंसिबिलिटी ज़रूरत), या एक अकेली स्टैटिक मेथड सचमुच आसान और पर्याप्त होगी? असली पॉलीमॉर्फिक एक्सटेंसिबिलिटी चाहिए तो Factory Method; एक सादा, केंद्रीकृत switch काफ़ी है तो "Simple Factory" (व्यावहारिक, GoF पैटर्न नहीं, और इंटरव्यू में यह खुलकर कहना बिल्कुल ठीक है)।

Explicit anti-triggers — do NOT reach for Factory Method when:

- There is, and will realistically only ever be, one concrete product type — this is premature abstraction with no payoff.
- What's actually needed is a *family* of related objects created consistently together — that's Abstract Factory, and conflating the two is one of the most common interview mix-ups for this pattern.
- The variation is really about *assembly steps*, not *which class* — many optional fields, fluent chaining — that's Builder's job, not Factory Method's.
- A plain static method with a `match / switch` would be simpler, has no real subclassing/extensibility requirement, and the team isn't planning to add types via polymorphism — reaching for full Factory Method machinery here is over-engineering; say so plainly rather than forcing the "proper" pattern.

स्पष्ट एंटी-ट्रिगर्स — Factory Method का इस्तेमाल न करें जब: - सिर्फ एक कॉन्क्रीट प्रोडक्ट टाइप है, और यथार्थवादी रूप से हमेशा रहेगी — यह बिना किसी फ़ायदे के समय-से-पहले अमूर्तन (premature abstraction) है। - असल में जो चाहिए वह है संबंधित ऑब्जेक्ट्स का एक परिवार, जो लगातार एक साथ बनाया जाए — यह Abstract Factory है, और दोनों को गड़मड़ करना इस पैटर्न की सबसे आम इंटरव्यू ग़लतफ़हमियों में से एक है। - बदलाव असल में असेंबली चरणों के बारे में है, कौन-सी क्लास के बारे में नहीं — कई वैकल्पिक फ़िल्ड्स, फ़्लुएंट चेनिंग — यह Builder का काम है, Factory Method का नहीं। - एक सादी स्टैटिक मेथड `match / switch` के साथ ज़्यादा आसान होगी, कोई असली सबक्लासिंग/एक्सटेंसिबिलिटी ज़रूरत नहीं है, और टीम पॉलीमॉर्फिज़्म के ज़रिए टाइप्स जोड़ने की योजना नहीं बना रही — यहाँ पूरी Factory Method मशीनरी का इस्तेमाल करना ओवर-इंजीनियरिंग है; यह खुलकर कहें, बजाय "सही" पैटर्न को ज़बरदस्ती थोपने के।

Part 4 — Cheat Sheet & Multi-Length Pitch

One-page cheat sheet:

Aspect	Summary
Problem solved	Client code that needs an object, but shouldn't be hard-wired to the concrete class of that object.
Mechanism	<code>Abstract Creator</code> declares a factory method returning a <code>Product</code> type; <code>ConcreteCreator</code> subclasses override it to return a specific <code>ConcreteProduct</code> .
Cost	An extra layer of classes/subclassing for what's sometimes a genuinely simple decision; can be over-engineering if the "who creates what" axis never actually varies.
Benefit	New product types can be added by adding a new <code>ConcreteCreator</code> , without touching existing client code — direct Open/Closed Principle payoff.
Common look-alike, not the same pattern	"Simple Factory" — one static method, one <code>switch</code> , no subclassing. Extremely common in real code; useful; just not this GoF pattern.
PHP-specific gotcha	Late static binding: inside an inherited factory method, <code>new static()</code> resolves to the calling subclass at runtime, while <code>new self()</code> always resolves to the class the method was originally written in — using the wrong one silently breaks the whole point of overriding the factory method in a subclass.

हिंदी अनुवाद / Hindi Translation:

पहलू	सार
हल की गई समस्या	क्लाइंट कोड को एक ऑब्जेक्ट चाहिए, लेकिन उसे उस ऑब्जेक्ट की कॉन्क्रीट क्लास से हार्ड-वायर्ड नहीं होना चाहिए।
तंत्र	ऐब्सट्रैक्ट <code>Creator</code> एक factory मेथड डिक्लेयर करता है जो <code>Product</code> टाइप लौटाता है; <code>ConcreteCreator</code> सबक्लासेज़ इसे ओवरराइड करके एक खास <code>ConcreteProduct</code> लौटाती हैं।
लागत	कभी-कभी सचमुच एक सादे फ़ैसले के लिए क्लासेज़/सबक्लासिंग की एक अतिरिक्त परत; अगर "कौन क्या बनाता है" धुरी कभी बदलती ही नहीं, तो यह ओवर-इंजीनियरिंग हो सकती है।

पहलू	सार
फ़ायदा	नए प्रोडक्ट टाइप्स को एक नया <code>ConcreteCreator</code> जोड़कर शामिल किया जा सकता है, बिना मौजूदा क्लाईंट कोड छुए — सीधा Open/Closed Principle फ़ायदा।
आम मिलता-जुलता, मगर वही पैटर्न नहीं	"Simple Factory" — एक स्टैटिक मेथड, एक <code>switch</code> , कोई सबक्लासिंग नहीं। असली कोड में बेहद आम; उपयोगी; बस यह GoF पैटर्न नहीं है।
PHP-विशिष्ट गड़बड़ी	Late static binding: एक इनहेरिटेड factory मेथड के अंदर, <code>new static()</code> रनटाइम पर कॉलिंग सबक्लास को रिज़ॉल्व करता है, जबकि <code>new self()</code> हमेशा उस क्लास को रिज़ॉल्व करता है जहाँ मेथड मूल रूप से लिखा गया था — ग़लत वाला इस्तेमाल करना, किसी सबक्लास में factory मेथड ओवरराइड करने का पूरा मक़सद चुपचाप तोड़ देता है।

30 seconds: "Factory Method lets a class declare that it needs to create some kind of object, without hard-coding which exact class that is — subclasses override the factory method to supply the specific type, so new types can be added by adding a new subclass instead of editing existing code."

30 सेकंड: "Factory Method एक क्लास को यह डिक्लेयर करने देता है कि उसे किसी तरह का ऑब्जेक्ट बनाना है, बिना यह हार्ड-कोड किए कि वह ठीक-ठीक कौन-सी क्लास है — सबक्लासेज़, factory मेथड को ओवरराइड करके खास टाइप देती हैं, इसलिए मौजूदा कोड एडिट करने के बजाय एक नई सबक्लास जोड़कर नई टाइप्स जोड़ी जा सकती हैं।"

1 minute: "Factory Method solves a specific problem: your code needs to create an object, but the exact concrete class of that object should be decided by something other than the code doing the creating — usually a subclass, sometimes configuration. You declare an abstract 'factory method' on a base `Creator` class that returns a `Product` interface type; each `ConcreteCreator` subclass overrides that one method to return its own specific `ConcreteProduct`. Everything else the `Creator` does is written against the `Product` interface, so it works correctly no matter which concrete type actually comes back. The payoff is Open/Closed Principle compliance — add a new product type by adding a new subclass, without touching the code that already works. The most common real-world mix-up is calling a single static method with an `if / switch` inside it 'a Factory' — that's a useful, simpler idiom, but it's not actually this GoF pattern, since there's no subclassing or polymorphic override involved."

1 मिनट: "Factory Method एक खास समस्या हल करता है: आपके कोड को एक ऑब्जेक्ट बनाना है, लेकिन उस ऑब्जेक्ट की ठीक-ठीक कॉन्क्रीट क्लास का फ़ैसला बनाने वाले कोड के अलावा किसी और चीज़ द्वारा होना चाहिए — आमतौर पर एक सबक्लास, कभी-कभी कॉन्फ़िगरेशन। आप एक बेस `Creator` क्लास पर एक ऐब्स्ट्रैक्ट 'factory मेथड' डिक्लेयर करते हैं जो `Product` इंटरफ़ेस टाइप लौटाता है; हर `ConcreteCreator` सबक्लास उस एक मेथड को ओवरराइड करके अपना खास `ConcreteProduct` लौटाती है। `Creator` जो बाक़ी सब करता है वह `Product` इंटरफ़ेस के खिलाफ़ लिखा जाता है, इसलिए यह सही ढंग से काम करता है चाहे असल में कोई भी कॉन्क्रीट टाइप वापस आए। फ़ायदा है Open/Closed Principle का पालन — मौजूदा काम कर रहे कोड को छुए बिना, एक नई सबक्लास जोड़कर एक नया प्रोडक्ट टाइप जोड़ें। सबसे आम असली-दुनिया की ग़लतफ़हमी है, एक अकेली स्टैटिक मेथड को जिसके अंदर `if / switch` हो, 'एक Factory' कहना — यह एक उपयोगी, सरल तरीक़ा है, लेकिन असल में यह GoF पैटर्न नहीं है, क्योंकि इसमें कोई सबक्लासिंग या पॉलीमॉर्फिक ओवरराइड शामिल नहीं है।"

3 minutes: adds — the Abstract Factory distinction (family of related creators vs. one creation method) with a concrete example of when you'd need the family; the Builder distinction

(assembly-steps vary vs. concrete-type varies); PHP's `new static()` vs `new self()` late-static-binding gotcha inside inherited factory methods; and a real framework example (Laravel's `Manager` base class family — `CacheManager`, `SessionManager`, `ChannelManager` — which resolves a concrete "driver" implementation by name via a `create{Name}Driver()` naming convention, which is Factory-Method- *adjacent* but technically closer to a registry-backed Simple Factory than textbook subclass-based Factory Method, worth being precise about the difference).

3 मिनट: इसमें जोड़ें — Abstract Factory का भेद (संबंधित creators का परिवार बनाम एक क्रिएशन मेथड) एक ठोस उदाहरण के साथ कि परिवार कब चाहिए होगा; Builder का भेद (असेंबली-चरण बदलते हैं बनाम कॉन्क्रीट-टाइप बदलती है); इनहेरिटेड factory मेथड्स के अंदर PHP का `new static()` बनाम `new self()` वाला late-static-binding गड़बड़ी; और एक असली फ्रेमवर्क उदाहरण (Laravel का `Manager` बेस क्लास परिवार — `CacheManager`, `SessionManager`, `ChannelManager` — जो `create{Name}Driver()` नामकरण परंपरा के ज़रिए नाम से एक कॉन्क्रीट "driver" इम्प्लीमेंटेशन रिज़ॉल्व करता है, जो Factory-Method-नज़दीकी है लेकिन तकनीकी रूप से टेक्स्टबुक सबक्लास-आधारित Factory Method से ज़्यादा एक रजिस्ट्री-समर्थित Simple Factory के करीब है, इस भेद को सटीक रखना ज़रूरी है)।

10 minutes: full pattern — everything above, plus walking the interviewer through a real example end-to-end (e.g., a `NotificationCreator` hierarchy for Email/SMS/Push), explicitly contrasting three implementations live (naive repeated conditionals → Simple Factory → full Factory Method) and stating exactly when each is the *right* amount of abstraction rather than always reaching for the most "proper" one, then closing with the SOLID story (strong OCP and DIP support) and the concurrency non-story (creation-time patterns like this one rarely have a meaningful thread-safety angle on their own, unlike Singleton — worth saying explicitly rather than forcing a non-existent concurrency discussion).

10 मिनट: पूरा पैटर्न — ऊपर की हर बात, साथ ही इंटरव्यूअर को एक असली उदाहरण शुरू से अंत तक दिखाना (जैसे, Email/SMS/Push के लिए `NotificationCreator` पदानुक्रम), लाइव तीन इम्प्लीमेंटेशन्स की तुलना करना (भोले-भाले दोहराई गई शर्तें → Simple Factory → पूरी Factory Method) और यह बताना कि हर एक कब सही मात्रा में अमूर्तन है, बजाय हमेशा सबसे "सही" वाली की तलाश करने के फिर SOLID कहानी (मज़बूत OCP और DIP समर्थन) और concurrency-न-कहानी (इस जैसे क्रिएशन-टाइम पैटर्न्स में शायद ही Singleton की तरह कोई सार्थक थ्रेड-सेफ्टी एंगल हो — इसे खुलकर कहना बेहतर है, बजाय एक ग़ैर-मौजूद concurrency चर्चा को ज़बरदस्ती थोपने के) के साथ समाप्त करना।

Part 5 — Timed Mock Drill

Prompt (45-60 minutes, live-coding style — India Tier-2/Tier-1-shaped per Part 2's data): *"Design the notification-creation layer for a backend that sends Email, SMS, and Push notifications. The system must support adding new channels (e.g., WhatsApp) in the future without modifying the code that already sends notifications. Implement it, then be ready to justify your design choice against two simpler alternatives an interviewer will push you toward."*

प्रॉम्प्ट (45-60 मिनट, लाइव-कोडिंग शैली): "एक बैकएंड के लिए नोटिफिकेशन-क्रिएशन लेयर डिज़ाइन करें जो *Email*, *SMS*, और *Push* नोटिफिकेशन्स भेजता है। सिस्टम को भविष्य में नए चैनल्स (जैसे *WhatsApp*) जोड़ना सपोर्ट करना चाहिए, बिना उस कोड को बदले जो पहले से नोटिफिकेशन्स भेज रहा है। इसे इम्प्लीमेंट करें, फिर दो सरल विकल्पों के मुकाबले अपने डिज़ाइन फ़ैसले को सही ठहराने के लिए तैयार रहें, जिनकी ओर इंटरव्यूअर आपको धकेलेगा।"

Time-boxed sub-steps: 1. **0-5 min** — Restate the requirement, confirm the real driver is "must be extensible to new types without touching existing client code" (an OCP requirement) rather than just "we need to send three kinds of messages today," and name the pattern out loud before writing code. 2. **5-20 min** — Implement: a `Notification` interface (`send(string $message): void`), concrete `EmailNotification` / `SmsNotification` / `PushNotification` classes, an abstract `NotificationCreator` with an abstract `createNotification(): Notification` factory method plus a shared `notify(string $message): void` template method that calls it, and concrete `EmailNotificationCreator` / `SmsNotificationCreator` / `PushNotificationCreator` subclasses. 3. **20-30 min** — Interviewer follow-up: "why not just a static method with a `switch` on a string type?" — correctly explain the trade-off: a Simple Factory is genuinely simpler and fine if there's no real subclassing/extensibility need, but Factory Method's subclass-per-type shape lets each `ConcreteCreator` also customize *surrounding* behavior (e.g., an `SmsNotificationCreator` might override retry logic differently from `EmailNotificationCreator`) in a way a single switch-based method can't cleanly express. 4. **30-40 min** — Interviewer follow-up: "add WhatsApp support — how many existing files change?" — correctly demonstrate zero changes to existing `Creator` / `Notification` code; only a new `WhatsAppNotification` + `WhatsAppNotificationCreator` pair is added. 5. **40-55 min** — Interviewer follow-up: "now we need to send an Email AND generate a matching in-app banner that must visually match — same 'family' every time. Does your design still fit?" — correctly recognize this has become an Abstract-Factory-shaped requirement (a *family* of related objects, not one), and explain how the design would need to change rather than forcing the existing Factory Method to cover it. 6. **55-60 min** — Wrap: state the design's trade-offs unprompted, including the honest note that this is more machinery than a two-notification-type system strictly needs today, justified specifically by the stated extensibility requirement.

समय-सीमित उप-चरण: 1. **0-5 मिनट** — ज़रूरत को दोहराएँ, पुष्टि करें कि असली वजह "बिना मौजूदा क्लाइंट कोड हुए नई टाइप्स के लिए एक्सटेंसिबल होना चाहिए" (एक OCP ज़रूरत) है, न कि सिर्फ़ "आज हमें तीन तरह के मैसेज भेजने हैं," और कोड लिखने से पहले पैटर्न का नाम ज़ोर से बताएँ। 2. **5-20 मिनट** — इम्प्लीमेंट करें: एक `Notification` इंटरफ़ेस, कॉन्क्रीट `EmailNotification` / `SmsNotification` / `PushNotification` क्लासेज़, एक ऐब्सट्रैक्ट `NotificationCreator` जिसमें एक ऐब्सट्रैक्ट `createNotification()` factory मेथड और उसे कॉल करने वाला एक साझा `notify()` टेम्पलेट मेथड हो, और कॉन्क्रीट `EmailNotificationCreator` / `SmsNotificationCreator` / `PushNotificationCreator` सबक्लासेज़। 3. **20-30 मिनट** — इंटरव्यूअर का फ़ॉलो-अप: "एक स्ट्रिंग टाइप पर `switch` वाली स्टैटिक मेथड क्यों नहीं?" — ट्रेड-ऑफ़ सही ढंग से समझाएँ: अगर कोई असली सबक्लासिंग/एक्सटेंसिबिलिटी ज़रूरत नहीं है तो Simple Factory सचमुच आसान और ठीक है, लेकिन Factory Method का प्रति-टाइप-सबक्लास आकार हर `ConcreteCreator` को आस-पास का व्यवहार भी कस्टमाइज़ करने देता है, ऐसे तरीके से जो एक अकेली `switch`-आधारित मेथड साफ़ तौर पर व्यक्त नहीं कर सकती। 4. **30-40 मिनट** — इंटरव्यूअर का फ़ॉलो-अप: "WhatsApp सपोर्ट जोड़ें — कितनी मौजूदा फ़ाइलें बदलती हैं?" — सही ढंग से दिखाएँ कि मौजूदा `Creator` / `Notification` कोड में शून्य बदलाव; सिर्फ़ एक नया `WhatsAppNotification` + `WhatsAppNotificationCreator` जोड़ा जाता है। 5. **40-55 मिनट** — इंटरव्यूअर का फ़ॉलो-अप: "अब हमें एक Email भेजना है AND एक मेल खाता इन-ऐप बैनर बनाना है जो दिखने में मेल खाना चाहिए — हर बार एक ही 'परिवार'। क्या आपका डिज़ाइन अब भी फ़िट बैठता है?" — सही ढंग से पहचानें कि यह अब Abstract-Factory-आकार की ज़रूरत बन गई है (संबंधित ऑब्जेक्ट्स का एक परिवार, एक नहीं), और समझाएँ कि डिज़ाइन को कैसे बदलना होगा, बजाय मौजूदा Factory Method को इसे कवर करने के लिए मजबूर करने के। 6. **55-60 मिनट** — समाप्ति: बिना पूछे डिज़ाइन के ट्रेड-ऑफ़्स बताएँ, इस ईमानदार टिप्पणी सहित कि यह आज

सख्ती से दो-नोटिफिकेशन-टाइप वाले सिस्टम की ज़रूरत से ज़्यादा मशीनरी है, जो खास तौर पर बताई गई एक्सटेंसिबिलिटी ज़रूरत से जायज़ ठहरती है।

Self-grading rubric: - [] Named the pattern and the specific requirement (extensibility, not just "creating objects") before writing code. - [] Correctly distinguished Factory Method from a Simple Factory when asked, without dismissing Simple Factory as "wrong." - [] Demonstrated adding a new type touches zero existing files. - [] Correctly recognized the "family of related objects" follow-up as an Abstract Factory shift, not a Factory Method extension. - [] Did not invent a concurrency/thread-safety discussion where none was warranted.

स्वयं-ग्रेडिंग रूब्रिक: - [] कोड लिखने से पहले पैटर्न और खास ज़रूरत (एक्सटेंसिबिलिटी, सिर्फ़ "ऑब्जेक्ट्स बनाना" नहीं) का नाम बताया। - [] पूछे जाने पर Factory Method को Simple Factory से सही ढंग से अलग किया, बिना Simple Factory को "ग़लत" कहे। - [] दिखाया कि एक नई टाइप जोड़ने से शून्य मौजूदा फ़ाइलें बदलती हैं। - [] "संबंधित ऑब्जेक्ट्स के परिवार" वाले फ़ॉलो-अप को सही ढंग से एक Abstract Factory बदलाव के तौर पर पहचाना, Factory Method एक्सटेंशन के तौर पर नहीं। - [] जहाँ कोई concurrency/थ्रेड-सेफ़्टी चर्चा जायज़ नहीं थी, वहाँ एक गढ़ी नहीं।

Part 6 — Pattern Recognition Drill

1. **"We need to generate a `PaymentReceipt` PDF, and the exact receipt layout/branding depends on which payment provider (Stripe, Razorpay, PayU) processed the transaction — new providers get added a few times a year."** → Factory Method — a `ReceiptCreator` base class with provider-specific subclasses, each overriding the one factory method; new providers add a subclass, no existing code changes. Not Abstract Factory (only one kind of object — a receipt — is being created, not a matched family), not Builder (the variation is *which* layout, not multi-step optional configuration).

हिंदी: हमें एक `PaymentReceipt` PDF बनाना है, और ठीक-ठीक रसीद लेआउट/ब्रांडिंग इस बात पर निर्भर करता है कि कौन-सा पेमेंट प्रोवाइडर (Stripe, Razorpay, PayU) लेन-देन को संसाधित करता है — साल में कुछ बार नए प्रोवाइडर जोड़े जाते हैं। → Factory Method — प्रोवाइडर-विशिष्ट सबक्लासेज़ वाली एक `ReceiptCreator` बेस क्लास, हर एक उस एक factory मेथड को ओवरराइड करती है; नए प्रोवाइडर एक सबक्लास जोड़ते हैं, कोई मौजूदा कोड नहीं बदलता। Abstract Factory नहीं (सिर्फ़ एक तरह का ऑब्जेक्ट — एक रसीद — बन रहा है, कोई मेल खाता परिवार नहीं), Builder नहीं (बदलाव कौन-सा लेआउट है, बहु-चरणीय वैकल्पिक कॉन्फ़िगरेशन नहीं)।

1. **"We're building a UI toolkit that must render either a full Windows-style widget set (buttons, checkboxes, scrollbars) or a full macOS-style widget set, and every widget on screen must match the same style consistently."** → Abstract Factory, not Factory Method — this is explicitly a *family* of related objects (button + checkbox + scrollbar) that must be created consistently together, the textbook Abstract Factory trigger.

हिंदी: हम एक UI टूलकिट बना रहे हैं जिसे या तो पूरा Windows-स्टाइल विजेट सेट या पूरा macOS-स्टाइल विजेट सेट रेंडर करना चाहिए, और स्क्रीन पर हर विजेट लगातार एक ही स्टाइल से मेल खाना चाहिए। → Abstract Factory, Factory Method नहीं —

यह साफ़ तौर पर संबंधित ऑब्जेक्ट्स का एक परिवार है (बटन + चेकबॉक्स + स्क्रॉलबार) जिसे लगातार एक साथ बनाया जाना चाहिए, टेक्स्टबुक Abstract Factory ट्रिगर।

1. **"Our `OrderCreator` class has one method that takes a `type` string and returns the right kind of `Order` via an `if / elseif` chain. It's called from exactly one place, and there are no plans to subclass or extend it."** → This is a Simple Factory, not Factory Method — useful, pragmatic, correctly scoped to the actual requirement; forcing subclassing/polymorphism onto it would be over-engineering with no payoff, and saying so plainly is the stronger interview answer than reflexively "upgrading" it.

हिंदी: हमारी `OrderCreator` क्लास में एक मेथड है जो एक `type` स्ट्रिंग लेती है और `if / elseif` चेन के ज़रिए सही तरह का `Order` लौटाती है। यह ठीक एक जगह से कॉल होती है, और इसे सबक्लास या एक्सटेंड करने की कोई योजना नहीं है। → यह एक Simple Factory है, Factory Method नहीं — उपयोगी, व्यावहारिक, असली ज़रूरत के हिसाब से सही ढंग से सीमित; इस पर सबक्लासिंग/पॉलीमॉर्फिज़्म थोपना बिना किसी फ़ायदे के ओवर-इंजीनियरिंग होगी, और यह खुलकर कहना इसे बिना सोचे "अपग्रेड" करने से बेहतर इंटरव्यू जवाब है।

1. **"We need to build a `SubscriptionPlan` object that has a base price, plus optional add-ons (extra storage, priority support, extra seats) that can be combined in many ways, and the construction reads awkwardly with a giant constructor."** → Builder, not Factory Method — the axis that varies here is *assembly steps and optional combinations*, not *which concrete class* gets instantiated.

हिंदी: हमें एक `SubscriptionPlan` ऑब्जेक्ट बनाना है जिसकी एक बेस कीमत है, साथ ही वैकल्पिक ऐड-ऑन्स (अतिरिक्त स्टोरेज, प्राथमिकता सपोर्ट, अतिरिक्त सीटें) जिन्हें कई तरीकों से जोड़ा जा सकता है, और एक विशाल कंस्ट्रक्टर के साथ निर्माण अजीब लगता है। → Builder, Factory Method नहीं — यहाँ जो धुरी बदलती है वह है असेंबली चरण और वैकल्पिक संयोजन, कौन-सी कॉन्क्रीट क्लास इंस्टेंशिएट होती है वह नहीं।

1. **"We have a `VehicleFactory` with a `createVehicle(string $type)` static method used in one place, but interviewers keep asking 'what pattern is this,' and the honest answer feels underwhelming."** → It's fine, and correct, to say "this is a Simple Factory — a common, useful idiom, but not formally a GoF pattern; if this needed to become extensible via subclassing without modifying this method, that's when it would evolve into Factory Method." Naming the honest distinction is a stronger signal than forcing a GoF label onto something that doesn't need one.

हिंदी: हमारे पास एक `VehicleFactory` है जिसमें एक `createVehicle(string $type)` स्टैटिक मेथड है, एक जगह इस्तेमाल होती है, लेकिन इंटरव्यूअर बार-बार पूछते हैं "यह कौन-सा पैटर्न है," और ईमानदार जवाब कमज़ोर लगता है। → यह कहना ठीक और सही है कि "यह एक Simple Factory है — एक आम, उपयोगी तरीका, लेकिन औपचारिक रूप से GoF पैटर्न नहीं; अगर इसे इस मेथड को बदले बिना सबक्लासिंग के ज़रिए एक्सटेंसिबल होना पड़े, तभी यह Factory Method में विकसित होगा।" ईमानदार भेद बताना, किसी ऐसी चीज़ पर GoF लेबल थोपने से बेहतर संकेत है जिसे उसकी ज़रूरत ही नहीं।

DEEP DIVE

Path map: `Fundamentals` → `Problem` → `Internals` → `Design` → `Implementation` → `Production` → `Trade-offs` → `Bugs` → `Interview Bank`.

पथ मानचित्र: बुनियाद → समस्या → आंतरिक कार्यप्रणाली → डिज़ाइन → इम्प्लीमेंटेशन → प्रोडक्शन → ट्रेड-ऑफ़्स → बग्स → इंटरव्यू बैंक।

Part 7 — Fundamentals

Definition: Factory Method defines an interface (or abstract method) for creating an object, but lets the classes that implement it decide which concrete class actually gets instantiated.

परिभाषा: Factory Method एक ऑब्जेक्ट बनाने के लिए एक इंटरफ़ेस (या ऐबस्ट्रैक्ट मेथड) डिफ़ाइन करता है, लेकिन यह तय करना उन क्लासेज़ पर छोड़ देता है जो इसे इम्प्लीमेंट करती हैं कि असल में कौन-सी कॉन्क्रीट क्लास इंस्टैंशिएट होगी।

Beginner framing: you have code that needs to create an object, but you don't want that code hard-wired to one specific class — so instead of calling `new SpecificClass()` directly, you call a method whose whole job is "give me a `Product`," and let something else (a subclass) decide what concrete `Product` that actually is.

शुरुआती स्तर की समझ: आपके पास ऐसा कोड है जिसे एक ऑब्जेक्ट बनाना है, लेकिन आप नहीं चाहते कि वह कोड किसी एक खास क्लास से हार्ड-वायर्ड हो — इसलिए सीधे `new SpecificClass()` कॉल करने के बजाय, आप एक ऐसी मेथड कॉल करते हैं जिसका पूरा काम है "मुझे एक `Product` दो," और यह तय करना किसी और चीज़ (एक सबक्लास) पर छोड़ देते हैं कि वह कॉन्क्रीट `Product` असल में क्या है।

Senior/staff framing: Factory Method is really about *deferring a decision* — specifically, the decision of "which concrete class" — from the code that uses an object to the code that creates it, and doing so through ordinary polymorphism rather than conditional logic. The deeper interview signal isn't "can you implement it" (most candidates can); it's whether you can correctly identify when this deferral is actually earning its keep (a genuine, ongoing extensibility need) versus when it's solving a problem that doesn't exist yet — over-applying this pattern to a two-case `if / else` that will never grow a third case is a real and commonly-flagged senior-level mistake, not a beginner one.

सीनियर/स्टाफ़ स्तर की समझ: Factory Method असल में एक फ़ैसले को टालने के बारे में है — खास तौर पर, "कौन-सी कॉन्क्रीट क्लास" वाले फ़ैसले को — ऑब्जेक्ट इस्तेमाल करने वाले कोड से हटाकर उसे बनाने वाले कोड तक, और यह शर्तों वाले लॉजिक के बजाय सामान्य पॉलीमॉर्फिज़्म के ज़रिए करना। गहरा इंटरव्यू संकेत यह नहीं है कि "क्या आप इसे इम्प्लीमेंट कर सकते हैं" (ज़्यादातर उम्मीदवार कर सकते हैं); यह है कि क्या आप सही ढंग से पहचान सकते हैं कि यह टालना कब असल में अपनी कीमत वसूल रहा है (एक असली, चल रही एक्सटेंसिबिलिटी ज़रूरत) बनाम कब यह एक ऐसी समस्या हल कर रहा है जो अभी मौजूद ही नहीं — इस पैटर्न को एक दो-केस वाले `if / else` पर ज़्यादा लगाना जो कभी तीसरा केस नहीं बढ़ाएगा, एक असली और अक्सर चिह्नित की जाने वाली सीनियर-स्तर की ग़लती है, शुरुआती-स्तर की नहीं।

Part 8 — The Engineering Problem & Refactoring Trigger

What code looks like before this pattern: a `NotificationSender` (or similar) has a method that branches on a type flag: `if ($channel === 'email') { $n = new EmailNotification(); } elseif ($channel === 'sms') { $n = new SmsNotification(); } elseif ($channel === 'push') { $n = new PushNotification(); }`. This exact block, or something close to it, tends to get copy-pasted into every place a notification needs to be created — a controller, a queued job, a CLI command.

यह पैटर्न लगाने से पहले कोड कैसा दिखता है: एक `NotificationSender` (या ऐसा ही कुछ) में एक मेथड है जो एक टाइप फ्लैग पर शाखाबद्ध होती है: `if ($channel === 'email') { $n = new EmailNotification(); } elseif ($channel === 'sms') { $n = new SmsNotification(); } elseif ($channel === 'push') { $n = new PushNotification(); }`। यह ठीक यही ब्लॉक, या इसके करीब कुछ, हर उस जगह कॉपी-पेस्ट होता जाता है जहाँ एक नोटिफिकेशन बनाना है — एक कंट्रोलर, एक क्यूड जॉब, एक CLI कमांड।

Why it breaks down at scale: every new channel means hunting down and editing every one of those duplicated conditional blocks. Miss one, and that code path silently keeps failing to support the new type — not a crash, just a quiet gap in coverage that surfaces as "why didn't this user get their WhatsApp notification" days later. This is a textbook Open/Closed Principle violation: adding a feature requires *modifying* existing, already-tested code rather than *adding* new code alongside it.

यह बड़े पैमाने पर क्यों टूटता है: हर नया चैनल मतलब उन दोहराए गए शर्तों वाले हर ब्लॉक को ढूँढ़ना और एडिट करना। एक छूट जाए, तो वह कोड पथ चुपचाप नई टाइप को सपोर्ट करने में विफल होता रहता है — कोई क्रैश नहीं, बस कवरेज में एक खामोश गैप जो दिनों बाद "इस यूज़र को उसका WhatsApp नोटिफिकेशन क्यों नहीं मिला" के तौर पर सामने आता है। यह एक टेक्स्टबुक Open/Closed Principle उल्लंघन है: एक फ़ीचर जोड़ने के लिए मौजूदा, पहले से टेस्ट किए गए कोड को बदलना पड़ता है, बजाय उसके साथ नया कोड जोड़ने के।

The code smell that should make an engineer reach for it: the same type-branching conditional, copy-pasted (or nearly so) across multiple files, each one needing a matching edit whenever a new type is added.

कोड स्मेल जो एक इंजीनियर को इसकी ओर ले जाना चाहिए: वही टाइप-शाखाबद्ध शर्त, कई फ़ाइलों में कॉपी-पेस्ट (या लगभग वैसी), हर एक को नई टाइप जोड़े जाने पर मेल खाता एडिट चाहिए।

Production-mindset questions: - *What production problem actually forced engineers toward this pattern?* — usually a missed-update incident: a new type was added in one place but a duplicated conditional elsewhere was forgotten, and the gap wasn't caught until a real user hit it. - *How would a senior engineer discover the requirement before it became a crisis?* — a code review flagging duplicated type-branching logic across files, or a grep for the same conditional pattern turning up three-plus near-identical copies. - *What metric would have shown it coming?* — a support/bug-ticket pattern of "channel X doesn't work in flow Y but does in flow Z" — a symptom of inconsistent coverage across duplicated creation logic. - *What alternatives would a competent engineer consider and reject first?* — "just be careful to update every copy" (doesn't scale with team size or codebase growth); a single centralized Simple Factory function (a real,

often-sufficient fix — rejected in favor of full Factory Method specifically when subclasses also need to vary *other* behavior alongside creation, not just the creation itself).

प्रोडक्शन-सोच वाले सवाल: - असल में किस प्रोडक्शन समस्या ने इंजीनियरों को इस पैटर्न की ओर मजबूर किया? — आमतौर पर एक छूटे-हुए-अपडेट की घटना: एक जगह नई टाइप जोड़ी गई लेकिन कहीं और एक दोहराई गई शर्त भूल गई, और यह गैप तब तक नहीं पकड़ा गया जब तक एक असली यूज़र इससे नहीं टकराया। - एक सीनियर इंजीनियर संकट बनने से पहले इस ज़रूरत को कैसे खोजेगा? — एक कोड रिव्यू जो फ़ाइलों में दोहराए गए टाइप-शाखाबद्ध लॉजिक को चिह्नित करे, या एक grep जो एक ही शर्त वाले पैटर्न की तीन-से-ज़्यादा लगभग-एक-जैसी कॉपियाँ ढूँढ़ निकाले। - कौन-सा मेट्रिक इसे आते हुए दिखाता? — "चैनल X फ़्लो Y में काम नहीं करता लेकिन फ़्लो Z में करता है" जैसा एक सपोर्ट/बग-टिकट पैटर्न — दोहराए गए क्रिएशन लॉजिक में असंगत कवरेज का एक लक्षण। - एक सक्षम इंजीनियर पहले किन विकल्पों पर विचार करके अस्वीकार करेगा? — "बस हर कॉपी अपडेट करने में सावधान रहो" (टीम के आकार या कोडबेस बढ़ोतरी के साथ स्केल नहीं करता); एक अकेला केंद्रीकृत Simple Factory फ़ंक्शन (एक असली, अक्सर पर्याप्त फ़िक्स — पूरी Factory Method के पक्ष में तब अस्वीकार किया जाता है जब सबक्लासेज़ को क्रिएशन के साथ-साथ अन्य व्यवहार भी बदलना हो, सिर्फ़ क्रिएशन नहीं)।

Part 9 — Internal Working

Concept level (language-agnostic): the mechanism is ordinary polymorphic method dispatch — nothing exotic. A `Creator` reference at runtime is actually pointing to some `ConcreteCreator` instance; calling the (possibly abstract) factory method resolves, via normal virtual-dispatch rules, to that specific subclass's override. The "magic" is entirely in how object-oriented dispatch already works — Factory Method is a *usage pattern* of polymorphism applied specifically to the moment of object creation, not a new language mechanism.

संकल्पना स्तर (भाषा-निरपेक्ष): तंत्र सामान्य पॉलीमॉर्फिक मेथड डिस्पैच है — कुछ भी विचित्र नहीं। रनटाइम पर एक `Creator` रेफ़रेंस असल में किसी `ConcreteCreator` इंस्टेंस की ओर इशारा कर रहा होता है; (शायद ऐब्सट्रेक्ट) factory मेथड कॉल करना, सामान्य वर्चुअल-डिस्पैच नियमों के ज़रिए, उस खास सबक्लास के ओवरराइड तक पहुँचता है। "जादू" पूरी तरह इसमें है कि ऑब्जेक्ट-ओरिएंटेड डिस्पैच पहले से कैसे काम करता है — Factory Method, पॉलीमॉर्फिज़्म का एक इस्तेमाल का तरीक़ा है जो खास तौर पर ऑब्जेक्ट-क्रिएशन के पल पर लगाया जाता है, कोई नया भाषा तंत्र नहीं।

PHP-specific mechanics — the one genuine gotcha worth knowing cold: PHP's **late static binding** (`static::` vs `self::`) matters a great deal inside factory methods that are inherited rather than fully overridden. If a base `Creator` class's factory method internally does `return new self();`, calling that method on a subclass instance will still construct the *base* class, not the subclass — because `self::` always resolves to the class where the code is physically written. Using `return new static();` instead resolves to whichever class the method was actually *called on* at runtime, which is almost always what a Factory-Method-style method actually needs. This single `self` vs. `static` distinction is a realistic, specific bug source in PHP factory implementations, and a sharp thing to bring up unprompted in an interview.

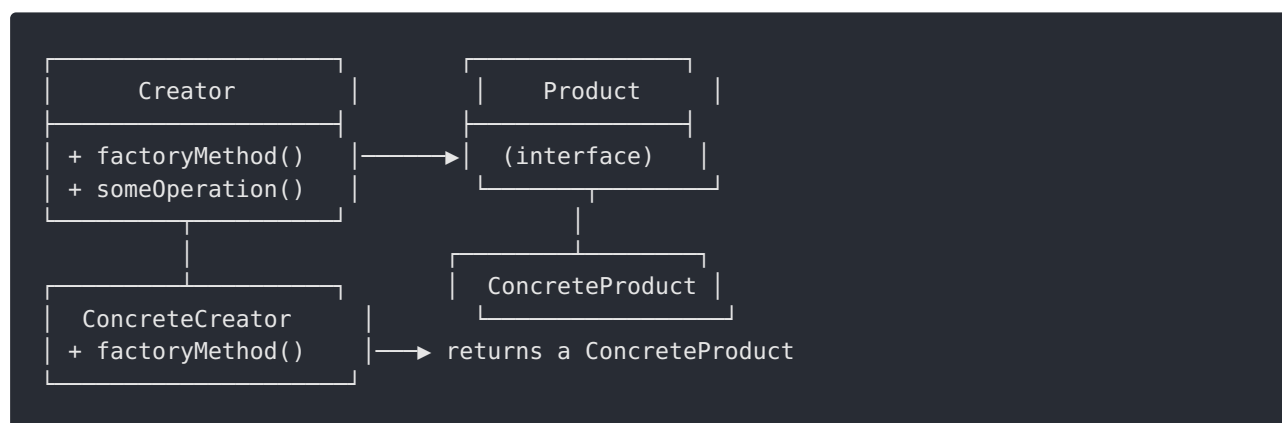
PHP-विशिष्ट कार्यप्रणाली — एक असली गड़बड़ी जिसे रट लेना ज़रूरी है: PHP की **late static binding** (`static::` बनाम `self::`) उन factory मेथड्स के अंदर बहुत मायने रखती है जो पूरी तरह ओवरराइड होने के बजाय इनहेरिट की जाती हैं। अगर एक बेस `Creator` क्लास की factory मेथड अंदर से `return new self();` करती है, तो एक सबक्लास इंस्टेंस पर उस मेथड को कॉल करने पर भी बेस क्लास ही बनेगी, सबक्लास नहीं — क्योंकि `self::` हमेशा उस क्लास को रिज़ॉल्व करता है जहाँ कोड भौतिक रूप से लिखा गया है। इसके बजाय `return new static();` इस्तेमाल करना उस क्लास को रिज़ॉल्व करता है जिस पर मेथड असल

में रनटाइम पर कॉल किया गया था, जो लगभग हमेशा वही है जो एक Factory-Method-शैली की मेथड को असल में चाहिए। यह अकेला `self` बनाम `static` भेद, PHP factory इम्प्लीमेंटेशन्स में एक यथार्थवादी, खास बग स्रोत है, और इंटरव्यू में बिना पूछे उठाने लायक एक तीखी बात है।

Part 10 — Components, UML & Language Mapping

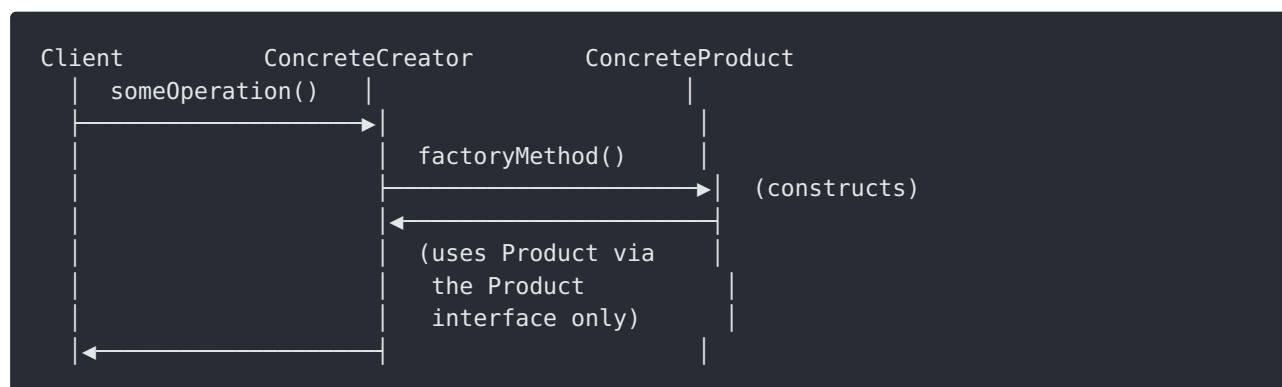
Roles: - **Product (interface):** the common type every concrete product implements. - **ConcreteProduct:** one specific implementation of Product. - **Creator (often abstract):** declares the factory method; may also contain other logic written entirely against the `Product` interface. - **ConcreteCreator:** overrides the factory method to return a specific `ConcreteProduct`.

भूमिकाएँ: - **Product (इंटरफ़ेस):** वह साझा टाइप जिसे हर कॉन्क्रीट प्रोडक्ट इम्प्लीमेंट करता है। - **ConcreteProduct:** Product का एक खास इम्प्लीमेंटेशन। - **Creator (अक्सर ऐब्स्ट्रैक्ट):** factory मेथड डिक्लेयर करता है; इसमें अन्य लॉजिक भी हो सकता है जो पूरी तरह `Product` इंटरफ़ेस के खिलाफ़ लिखा गया हो। - **ConcreteCreator:** एक खास `ConcreteProduct` लौटाने के लिए factory मेथड को ओवरराइड करता है।



Sequence (why this is worth drawing on a whiteboard):

अनुक्रम (यह व्हाइटबोर्ड पर बनाने लायक क्यों है):



Language mapping for the core mechanism:

Language	How "let a subclass decide the concrete type" is typically achieved
PHP 8.3	Abstract method on a base class, overridden per subclass; watch the <code>self</code> vs. <code>static</code> late-static-binding gotcha from Part 9.
Java	Abstract method, identical shape to PHP/GoF's original textbook form — this is the language the original GoF examples are closest to.
Python	Often simpler in practice — a module-level function or a <code>@classmethod</code> returning different types based on a parameter, since Python's flexibility makes a full class hierarchy less necessary for simple cases.
Go	No inheritance, so "Factory Method" in Go usually means a constructor <i>function</i> returning an interface type, with the concrete struct chosen by a parameter or by which package-level function was called — same intent, different mechanism (composition/functions instead of subclassing).
TypeScript/Node.js	Abstract class with an abstract method, close to the PHP/Java shape, though a parameterized factory function is at least as common in idiomatic TypeScript.

हिंदी सार (भाषा-मानचित्रण): PHP 8.3 में — बेस क्लास पर ऐब्स्ट्रैक्ट मेथड, हर सबक्लास में ओवरराइड; Part 9 की `self` बनाम `static` गड़बड़ी पर ध्यान दें। Java में — ऐब्स्ट्रैक्ट मेथड, PHP/GoF के मूल टेक्स्टबुक रूप जैसा ही। Python में — अक्सर व्यवहार में सरल: एक मॉड्यूल-स्तर का फ़ंक्शन या एक `@classmethod` जो पैरामीटर के आधार पर अलग टाइप्स लौटाता है। Go में — कोई इनहेरिटेन्स नहीं, इसलिए "Factory Method" का मतलब आमतौर पर एक कंस्ट्रक्टर फ़ंक्शन होता है जो एक इंटरफ़ेस टाइप लौटाता है। TypeScript/Node.js में — एक ऐब्स्ट्रैक्ट मेथड वाली ऐब्स्ट्रैक्ट क्लास, PHP/Java के आकार के करीब, हालाँकि एक पैरामीटराइज़्ड factory फ़ंक्शन भी उतना ही आम है।

Part 11 — Implementation Overview (PHP/Laravel/Node)

The companion `Factory.php` file walks through three stages: repeated inline conditionals (the "before" state from Part 8), a Simple Factory refactor, and a full Factory Method refactor with a `NotificationCreator` hierarchy — plus a short demonstration of the `self` vs. `static` gotcha from Part 9.

साथी `Factory.php` फ़ाइल तीन चरणों से गुज़रती है: दोहराई गई इनलाइन शर्तें (Part 8 की "पहले" वाली स्थिति), एक Simple Factory रीफ़ैक्टर, और एक `NotificationCreator` पदानुक्रम वाला पूरा Factory Method रीफ़ैक्टर — साथ ही Part 9 की `self` बनाम `static` गड़बड़ी का एक छोटा प्रदर्शन।

Where this pattern genuinely does — and doesn't — show up in real framework internals, verified against current source rather than assumed:

यह पैटर्न असली फ़्रेमवर्क आंतरिक कार्यप्रणाली में सचमुच कहाँ दिखता है — और कहाँ नहीं — मौजूदा स्रोत के खिलाफ़ सत्यापित, न कि मान लिया गया:

Laravel's `Illuminate\Support\Manager` abstract class — the shared base behind `CacheManager`, `SessionManager`, and `Illuminate\Notifications\ChannelManager`, among others — is the framework mechanism most PHP engineers reach for when this topic comes up, and it's worth

being precise about what it actually is. Fetched and verified against current source:

`Manager::driver($name)` resolves a requested driver, and if it hasn't been built yet, calls a protected `createDriver($name)` method, which in turn looks for a method on `$this` named by convention — `create{StudlyName}Driver()` — and calls it, caching the result. This is **not** textbook subclass-based Factory Method — there's no abstract factory method that each subclass overrides to return one specific type. It's closer to a **Simple Factory with naming-convention-based method dispatch**, bundled with per-instance caching (so it also does a bit of what a registry does). It's a genuinely useful, real production pattern — just worth naming accurately rather than calling it "Laravel's implementation of Factory Method," which overstates the resemblance. This is exactly the kind of precise distinction (Part 1's "closest confused patterns") that separates a candidate who's memorized the GoF definition from one who's actually mapped it onto real code.

Laravel की `Illuminate\Support\Manager` ऐब्सट्रैक्ट क्लास — `CacheManager`, `SessionManager`, और `Illuminate\Notifications\ChannelManager` जैसों के पीछे की साझा बेस — वह फ्रेमवर्क तंत्र है जिसकी ओर ज़्यादातर PHP इंजीनियर इस विषय के आने पर पहुँचते हैं, और यह बताना ज़रूरी है कि यह असल में क्या है। मौजूदा स्रोत के खिलाफ़ फ़ेच और सत्यापित किया गया: `Manager::driver($name)` एक माँगा गया driver रिज़ॉल्व करता है, और अगर यह अभी तक नहीं बना है, तो एक प्रोटेक्टेड `createDriver($name)` मेथड कॉल करता है, जो बदले में `$this` पर एक परंपरा से नामित मेथड — `create{StudlyName}Driver()` — ढूँढ़ता है और उसे कॉल करता है, नतीजे को कैश करते हुए। यह टेक्स्टबुक सबक्लास-आधारित Factory Method **नहीं** है — कोई ऐब्सट्रैक्ट factory मेथड नहीं है जिसे हर सबक्लास एक खास टाइप लौटाने के लिए ओवरराइड करे। यह नामकरण-परंपरा-आधारित मेथड डिस्पैच वाली एक **Simple Factory** के ज़्यादा करीब है, प्रति-इंस्टेंस कैशिंग के साथ बंडल्ड (इसलिए यह थोड़ा वह भी करता है जो एक रजिस्ट्री करती है)। यह एक सचमुच उपयोगी, असली प्रोडक्शन पैटर्न है — बस इसे सही ढंग से नाम देना ज़रूरी है, इसे "Laravel का Factory Method इम्प्लीमेंटेशन" कहने के बजाय, जो समानता को बढ़ा-चढ़ाकर बताता है। यह ठीक वैसा सटीक भेद है (Part 1 के "सबसे मिलते-जुलते भ्रामक पैटर्न्स") जो एक ऐसे उम्मीदवार को अलग करता है जिसने GoF परिभाषा रट ली है, उससे जिसने इसे असली कोड पर मैप किया है।

Node.js/Express codebases rarely implement a class-hierarchy version of this pattern at all — a parameterized factory *function* (`function createNotification(type) { switch(type) {...} }`) does the same job with far less ceremony, which is itself worth naming in an interview as the idiomatic JS/TS equivalent rather than forcing a class hierarchy where the language doesn't need one.

Node.js/Express कोडबेस शायद ही कभी इस पैटर्न का क्लास-पदानुक्रम वर्शन इम्प्लीमेंट करते हैं — एक पैरामीटराइज़्ड factory फ़ंक्शन (`function createNotification(type) { switch(type) {...} }`) कहीं कम औपचारिकता के साथ वही काम करता है, जो खुद इंटरव्यू में मुहावरेदार JS/TS समकक्ष के तौर पर नाम देने लायक है, बजाय वहाँ क्लास पदानुक्रम थोपने के जहाँ भाषा को इसकी ज़रूरत नहीं।

Part 12 — Where This Shows Up in Production

Scenario 1 — Multi-channel notification system (Razorpay/Postman-style, per Part 2's India Tier-2 data): a `NotificationCreator` hierarchy with `EmailNotificationCreator`, `SmsNotificationCreator`, `PushNotificationCreator` subclasses, each also customizing retry/

backoff behavior specific to that channel — the concrete justification for Factory Method over a Simple Factory, since more than just "which class" varies per subclass.

परिदृश्य 1 — मल्टी-चैनल नोटिफिकेशन सिस्टम (Razorpay/Postman-शैली): `EmailNotificationCreator`, `SmsNotificationCreator`, `PushNotificationCreator` सबक्लासेज़ वाला एक `NotificationCreator` पदानुक्रम, हर एक उस चैनल के लिए खास retry/backoff व्यवहार भी कस्टमाइज़ करता है — Simple Factory के मुकाबले Factory Method का ठोस औचित्य, क्योंकि हर सबक्लास में सिर्फ़ "कौन-सी क्लास" से ज़्यादा बदलता है।

Scenario 2 — Payment-method object creation (fintech-shaped, per Part 2's Tier-1/Tier-2 data): a `PaymentMethodCreator` hierarchy producing `CreditCardPayment`, `UpiPayment`, `WalletPayment` objects, where new payment rails get added periodically without touching the checkout flow that consumes them — a genuine, ongoing extensibility need rather than a one-off decision.

परिदृश्य 2 — पेमेंट-मैथड ऑब्जेक्ट क्रिएशन (फ़िनटेक-आकार): `CreditCardPayment`, `UpiPayment`, `WalletPayment` ऑब्जेक्ट्स बनाने वाला एक `PaymentMethodCreator` पदानुक्रम, जहाँ उसे इस्तेमाल करने वाले चेकआउट प्रलो को छुए बिना समय-समय पर नए पेमेंट रेल जोड़े जाते हैं — एक असली, चल रही एक्सटेंसिबिलिटी ज़रूरत, एक बार का फैसला नहीं।

Scenario 3 — Vehicle/parking-slot allocation (Parking Lot System, explicitly named in the frequency guide's classic-problems list): a `VehicleCreator` hierarchy or Simple Factory (depending on whether subclasses need to vary allocation *behavior*, not just vehicle type) producing `Car`, `Bike`, `Truck` objects with different slot-size requirements.

परिदृश्य 3 — वाहन/पार्किंग-स्लॉट आवंटन (Parking Lot System): एक `VehicleCreator` पदानुक्रम या Simple Factory (इस पर निर्भर कि सबक्लासेज़ को आवंटन व्यवहार बदलना है या नहीं, सिर्फ़ वाहन टाइप नहीं) जो अलग-अलग स्लॉट-आकार ज़रूरतों वाले `Car`, `Bike`, `Truck` ऑब्जेक्ट्स बनाता है।

Microservices-usage table:

Component	Typically Factory-Method-shaped?	Why
Multi-channel notification dispatch	Yes, when per-channel behavior varies beyond just creation	Genuine subclass-level extensibility need
Payment-method/gateway instantiation	Often	New rails added periodically, existing checkout code shouldn't change
Parser/deserializer selection by content-type	Often, or Simple Factory	Depends on whether parser <i>behavior</i> varies beyond construction
A one-off object with no planned variants	No	Classic over-engineering trap — plain constructor is correct
A family of visually-matched UI components	No — Abstract Factory instead	The "family" requirement is the tell

हिंदी सार (माइक्रोसर्विसेज़-इस्तेमाल टेबल): मल्टी-चैनल नोटिफिकेशन डिस्पैच — हाँ, जब प्रति-चैनल व्यवहार सिर्फ़ क्रिएशन से आगे बदलता है (असली सबक्लास-स्तर एक्सटेंसिबिलिटी ज़रूरत)। पेमेंट-मैथड/गेटवे इंस्टैंशिएशन — अक्सर (नए रेल समय-समय पर जुड़ते हैं,

मौजूदा चेकआउट कोड नहीं बदलना चाहिए)। कंटेंट-टाइप से पार्सर/डीसीरियलाइज़र चुनना — अक्सर, या Simple Factory (इस पर निर्भर कि पार्सर व्यवहार निर्माण से आगे बदलता है या नहीं)। बिना योजनाबद्ध वेरिएंट वाला एक बार का ऑब्जेक्ट — नहीं (क्लासिक ओवर-इंजीनियरिंग जाल — सादा कंस्ट्रक्टर सही है)। दिखने में मेल खाते UI कंपोनेंट्स का एक परिवार — नहीं, इसके बजाय Abstract Factory ("परिवार" ज़रूरत ही संकेत है)।

Architecture Decision Record — choosing Factory Method over a Simple Factory for

`NotificationCreator` :

आर्किटेक्चर डिज़ीज़न रिकॉर्ड — `NotificationCreator` के लिए Simple Factory के मुकाबले Factory Method चुनना:

- **Context:** the notification system supported Email and SMS via a single static `NotificationFactory::make($type)` method with a `match` expression. A new requirement arrived: SMS notifications needed provider-specific retry/backoff logic that Email didn't need, and a WhatsApp channel was already planned for next quarter with its own delivery-confirmation webhook handling.
- **संदर्भ:** नोटिफिकेशन सिस्टम एक `match` एक्सप्रेसन वाली एक अकेली स्टैटिक `NotificationFactory::make($type)` मेथड के ज़रिए Email और SMS सपोर्ट करता था। एक नई ज़रूरत आई: SMS नोटिफिकेशन्स को प्रोवाइडर-विशिष्ट retry/backoff लॉजिक चाहिए था जो Email को नहीं चाहिए था, और एक WhatsApp चैनल अगली तिमाही के लिए पहले से योजनाबद्ध था, अपनी खुद की डिलीवरी-पुष्टि वेबहुक हैंडलिंग के साथ।
- **Decision:** refactor to a full `NotificationCreator` class hierarchy, with each `ConcreteCreator` subclass owning both its factory method and its channel-specific surrounding behavior (retry policy, delivery confirmation handling).
- **फ़ैसला:** एक पूरी `NotificationCreator` क्लास पदानुक्रम में रीफैक्टर करें, हर `ConcreteCreator` सबक्लास अपनी factory मेथड और अपने चैनल-विशिष्ट आस-पास के व्यवहार (retry नीति, डिलीवरी पुष्टि हैंडलिंग) दोनों की मालिक हो।
- **Consequences:** each channel's creation logic and its behavioral quirks now live together in one cohesive subclass, instead of creation living in a central switch while behavior lives scattered elsewhere; adding WhatsApp next quarter means adding one new subclass, touching zero existing files.
- **नतीजे:** हर चैनल का क्रिएशन लॉजिक और उसकी व्यवहारगत विशेषताएँ अब एक साथ एक एकीकृत सबक्लास में रहती हैं, न कि क्रिएशन एक केंद्रीय switch में और व्यवहार कहीं और बिखरा हुआ; अगली तिमाही WhatsApp जोड़ने का मतलब एक नई सबक्लास जोड़ना है, शून्य मौजूदा फ़ाइलों को छुए बिना।
- **Alternatives considered:** keeping the Simple Factory and handling per-channel retry logic via a separate strategy object composed in afterward — rejected as workable but ultimately just re-inventing what Factory Method already gives for free, at the cost of an extra layer of indirection.
- **विचार किए गए विकल्प:** Simple Factory को बनाए रखना और प्रति-चैनल retry लॉजिक को बाद में जोड़े गए एक अलग strategy ऑब्जेक्ट के ज़रिए संभालना — व्यावहारिक होते हुए भी अस्वीकार किया गया, क्योंकि यह अंततः वही फिर से बना रहा है जो Factory Method पहले से मुफ्त में देता है, एक अतिरिक्त अप्रत्यक्षता परत की कीमत पर।
- **Trade-offs:** the team accepted more classes and more structural ceremony than the Simple Factory version, justified specifically by the *combined* creation-plus-behavior

variation — a team should not make this same trade if only the creation step varies and behavior stays uniform across types.

- **ट्रेड-ऑफ़्स:** टीम ने Simple Factory वर्शन से ज़्यादा क्लासेज़ और ज़्यादा संरचनात्मक औपचारिकता स्वीकार की, खास तौर पर संयुक्त क्रिएशन-प्लस-व्यवहार बदलाव से जायज़ ठहराई गई — अगर सिर्फ़ क्रिएशन चरण बदलता है और व्यवहार सभी टाइप्स में एक जैसा रहता है, तो एक टीम को यही ट्रेड-ऑफ़ नहीं करना चाहिए।

Part 13 — Field Notes (Simulated Production Experience)

Rehearsal scaffold, not a script — personalize with real project details before using as an actual interview answer, or present it plainly as illustrative rather than personal history.

रिहर्सल ढाँचा, कोई स्क्रिप्ट नहीं — असली इंटरव्यू जवाब के तौर पर इस्तेमाल करने से पहले असली प्रोजेक्ट विवरणों के साथ निजीकृत करें, या इसे व्यक्तिगत इतिहास के बजाय स्पष्ट रूप से उदाहरणात्मक बताकर पेश करें।

"On a payments-adjacent team, we had a `PaymentMethodFactory::create($type)` static method that had grown to a dozen `case` branches over two years. It technically worked, but every new payment rail meant editing that one increasingly unwieldy method, and two different engineers had, on separate occasions, accidentally broken an unrelated `case` branch while adding a new one — classic blast-radius risk from cramming unrelated logic into one big switch. We refactored to a `PaymentMethodCreator` hierarchy, one subclass per rail, and the immediate win wasn't really performance or elegance — it was that a new payment rail became a self-contained PR touching exactly one new file, reviewable in isolation, with zero risk of breaking an existing rail's logic by accident. The honest caveat I'd add for an interview: this was worth doing specifically because we had a dozen types and a proven pattern of frequent additions — I wouldn't make this same call for a system with two stable types and no growth in sight."

"एक पेमेंट्स-नज़दीकी टीम में, हमारे पास एक `PaymentMethodFactory::create($type)` स्टैटिक मेथड था जो दो साल में एक दर्जन `case` शाखाओं तक बढ़ गया था। यह तकनीकी रूप से काम करता था, लेकिन हर नया पेमेंट रेल मतलब उस एक बढ़ती हुई बोझिल मेथड को एडिट करना, और अलग-अलग मौकों पर दो अलग इंजीनियरों ने गलती से एक असंबंधित `case` शाखा तोड़ दी थी जब वे एक नई जोड़ रहे थे — असंबंधित लॉजिक को एक बड़े switch में ढूँढने से आने वाला क्लासिक ब्लास्ट-रेडियस जोखिम। हमने `PaymentMethodCreator` पदानुक्रम में रीफैक्टर किया, हर रेल के लिए एक सबक्लास, और तुरंत फ़ायदा असल में परफ़ॉर्मेंस या सुरुचि नहीं था — यह था कि एक नया पेमेंट रेल ठीक एक नई फ़ाइल छूने वाला एक स्वयं-निहित PR बन गया, अलग से रिव्यू करने लायक, किसी मौजूदा रेल के लॉजिक को गलती से तोड़ने के शून्य जोखिम के साथ। इंटरव्यू के लिए मैं जो ईमानदार चेतावनी जोड़ूँगा: यह करना खास तौर पर इसलिए सार्थक था क्योंकि हमारे पास एक दर्जन टाइप्स थीं और बार-बार जोड़े जाने का एक सिद्ध पैटर्न था — मैं दो स्थिर टाइप्स और कोई बढ़ोतरी न दिखने वाले सिस्टम के लिए यही फैसला नहीं लेता।"

Part 14 — Analogies & Architecture Fit

Analogies: - **Pizza-chain franchise** (Part 1's memory hook) — shared process, franchise-specific product. Best single analogy for "shared behavior, varying creation." - **A publishing house with regional imprints** — the parent company's editorial/distribution process (the `Creator`'s ordinary methods) is identical everywhere, but each regional imprint (`ConcreteCreator`) decides which specific format to produce (hardcover here, a regional-

language edition there) via its own `produceBook()` (the factory method). - **A hospital's triage desk routing to the right specialist** — weaker analogy, worth naming as weak: routing to an *existing* specialist is closer to Strategy (selecting among existing behaviors) than to Factory Method (which is specifically about *object creation*). Useful to have ready to correct an interviewer who offers it.

उपमाएँ: - **पिज़्ज़ा-चेन फ्रेंचाइज़ी** (Part 1 की याद रखने की तरकीब) — साझा प्रक्रिया, फ्रेंचाइज़ी-विशिष्ट प्रोडक्ट। "साझा व्यवहार, बदलता क्रिएशन" के लिए सबसे अच्छी अकेली उपमा। - **क्षेत्रीय इम्प्रिंट्स वाला एक प्रकाशन गृह** — पैरेंट कंपनी की संपादकीय/वितरण प्रक्रिया (`Creator` के सामान्य मेथड्स) हर जगह एक जैसी है, लेकिन हर क्षेत्रीय इम्प्रिंट (`ConcreteCreator`) अपने `produceBook()` (factory मेथड) के ज़रिए तय करता है कि कौन-सा खास फ़ॉर्मेट बनाना है (यहाँ हार्डकवर, वहाँ एक क्षेत्रीय-भाषा संस्करण)। - **एक अस्पताल का ट्राइएज डेस्क सही विशेषज्ञ की ओर भेजना** — कमज़ोर उपमा, इसे कमज़ोर बताना ज़रूरी है: एक मौजूदा विशेषज्ञ की ओर भेजना Strategy (मौजूदा व्यवहारों में से चुनना) के ज़्यादा करीब है, Factory Method (जो खास तौर पर ऑब्जेक्ट क्रिएशन के बारे में है) के नहीं। एक इंटरव्यूअर को सुधारने के लिए तैयार रखने लायक, जो इसे पेश करे।

Architecture fit: - **Clean/Hexagonal/Onion:** a very natural fit at the boundary between application/use-case layer and infrastructure — a `Creator` deciding which concrete adapter/gateway implementation to instantiate based on configuration is a common, legitimate use, keeping the decision out of the domain layer entirely. - **DDD:** less central than in infrastructure-heavy layers — Factory Method (and its close cousin, a domain-level "Factory" concept in DDD terminology, which is a related-but-distinct idea about encapsulating complex Aggregate construction) can help construct Aggregates whose concrete type varies, but most Aggregate construction is simpler than this pattern requires. - **Event-driven architecture:** a message/event *deserializer* selected by event-type is a common, legitimate Factory-Method-or-Simple-Factory-shaped need — worth naming explicitly since "polymorphic dispatch by message type" is a frequent real system-design element. - **CQRS:** no strong, direct connection — worth stating plainly. - **Cloud-native/Kubernetes:** no meaningful connection at the object-creation level; not worth forcing.

आर्किटेक्चर फ़िट: - **Clean/Hexagonal/Onion:** एप्लिकेशन/यूज़-केस लेयर और इन्फ्रास्ट्रक्चर के बीच की सीमा पर एक बहुत स्वाभाविक फ़िट — एक `Creator` जो कॉन्फ़िगरेशन के आधार पर तय करता है कि कौन-सा कॉन्क्रीट adapter/gateway इम्प्लीमेंटेशन इंस्टैंशिएट करना है, एक आम, वैध इस्तेमाल है, फैसले को डोमेन लेयर से पूरी तरह बाहर रखते हुए। - **DDD:** इन्फ्रास्ट्रक्चर-भारी लेयर्स से कम केंद्रीय — Factory Method (और इसका करीबी रिश्तेदार, DDD शब्दावली में एक डोमेन-स्तरीय "Factory" संकल्पना, जो जटिल Aggregate निर्माण को एनकैप्सुलेट करने का एक संबंधित-मगर-अलग विचार है) उन Aggregates को बनाने में मदद कर सकता है जिनकी कॉन्क्रीट टाइप बदलती है, लेकिन ज़्यादातर Aggregate निर्माण इस पैटर्न की ज़रूरत से सरल होता है। - **इवेंट-ड्रिवन आर्किटेक्चर:** इवेंट-टाइप से चुना गया एक मैसेज/इवेंट डीसीरियलाइज़र एक आम, वैध Factory-Method-या-Simple-Factory-आकार की ज़रूरत है — खुलकर नाम देने लायक क्योंकि "मैसेज टाइप से पॉलीमॉर्फिक डिस्पैच" एक आम असली सिस्टम-डिज़ाइन तत्व है। - **CQRS:** कोई मज़बूत, सीधा संबंध नहीं — खुलकर कहने लायक। - **Cloud-native/Kubernetes:** ऑब्जेक्ट-क्रिएशन स्तर पर कोई सार्थक संबंध नहीं; थोपने लायक नहीं।

✓ **Before you move on:** (1) Which analogy is actually closer to Strategy than Factory Method, and why? (2) Name one architecture style where this pattern has no meaningful connection, stated plainly rather than forced.

✓ **आगे बढ़ने से पहले:** (1) कौन-सी उपमा असल में Factory Method से ज़्यादा Strategy के करीब है, और क्यों? (2) एक आर्किटेक्चर शैली बताएँ जहाँ इस पैटर्न का कोई सार्थक संबंध नहीं है, खुलकर कहा गया, थोपा नहीं गया।

Part 15 — SOLID, Performance & Concurrency

SOLID: this is one of the strongest, most defensible SOLID stories among the creational patterns, worth stating with confidence rather than hedging. **Open/Closed Principle** is the headline: new product types are added via new subclasses, with zero modification to existing, already-tested `Creator` /client code — the textbook OCP payoff. **Dependency Inversion** is supported directly: client code depends only on the abstract `Product` / `Creator` types, never on concrete classes. **Single Responsibility** is mildly supported — creation logic is cleanly separated from the logic that uses the created object — though this is a secondary benefit, not the pattern's main point. **Liskov Substitution** matters at the `Product` level: every `ConcreteProduct` must be genuinely substitutable wherever `Product` is expected, or client code written against the interface breaks in subtle ways. **Interface Segregation** has no strong, direct connection worth forcing.

SOLID: यह क्रिएशनल पैटर्न्स में सबसे मज़बूत, सबसे बचाव-योग्य SOLID कहानियों में से एक है, हिचकिचाहट के बजाय भरोसे के साथ कहने लायक। **Open/Closed Principle** मुख्य बात है: नए प्रोडक्ट टाइप्स नई सबक्लासेज़ के ज़रिए जोड़े जाते हैं, मौजूदा, पहले से टेस्ट किए गए `Creator` /क्लाइंट कोड में शून्य बदलाव के साथ — टेक्स्टबुक OCP फ़ायदा। **Dependency Inversion** सीधे समर्थित है: क्लाइंट कोड सिर्फ़ ऐबस्ट्रैक्ट `Product` / `Creator` टाइप्स पर निर्भर करता है, कभी कॉन्क्रीट क्लासेज़ पर नहीं। **Single Responsibility** हल्के तौर पर समर्थित है — क्रिएशन लॉजिक, बनाए गए ऑब्जेक्ट को इस्तेमाल करने वाले लॉजिक से साफ़ तौर पर अलग है — हालाँकि यह एक द्वितीयक फ़ायदा है, पैटर्न का मुख्य बिंदु नहीं। **Liskov Substitution** `Product` स्तर पर मायने रखता है: हर `ConcreteProduct` को हर जगह वाकई प्रतिस्थापन-योग्य होना चाहिए जहाँ `Product` अपेक्षित है, वरना इंटरफ़ेस के खिलाफ़ लिखा गया क्लाइंट कोड सूक्ष्म तरीकों से टूटता है। **Interface Segregation** का कोई मज़बूत, सीधा संबंध नहीं जिसे थोपा जाए।

Performance: genuinely neutral-to-positive in almost all cases — the pattern adds one virtual method call's worth of indirection over calling `new` directly, which is negligible in essentially every real system. There is no meaningful performance story here, and claiming a significant one would be a red flag, not a strength, in an interview answer.

परफ़ॉर्मेंस: लगभग सभी मामलों में सचमुच तटस्थ-से-सकारात्मक — यह पैटर्न सीधे `new` कॉल करने के मुकाबले एक वर्चुअल मेथड कॉल जितनी अप्रत्यक्षता जोड़ता है, जो अनिवार्य रूप से हर असली सिस्टम में नगण्य है। यहाँ कोई सार्थक परफ़ॉर्मेंस कहानी नहीं है, और एक महत्वपूर्ण कहानी का दावा करना इंटरव्यू जवाब में एक ताक़त नहीं, एक चेतावनी संकेत होगा।

Concurrency: also genuinely neutral in the vast majority of cases — unlike Singleton, Factory Method's core mechanism (polymorphic dispatch to create a new, independent object) has no shared-mutable-state race condition to guard against by default, since each call typically produces a fresh, unshared object. The one place concurrency becomes relevant is if a `ConcreteCreator` *itself* holds and mutates shared state across calls (e.g., an internal counter or cache) — at that point the concurrency reasoning is really about that specific shared state, not about Factory Method as a pattern. Correctly saying "there's no meaningful concurrency story here, and here's precisely why" is a stronger answer than manufacturing one.

Concurrency: ज़्यादातर मामलों में भी सचमुच तटस्थ — Singleton के उलट, Factory Method के मुख्य तंत्र (एक नया, स्वतंत्र ऑब्जेक्ट बनाने के लिए पॉलीमॉर्फिक डिस्पैच) में डिफ़ॉल्ट रूप से बचाने के लिए कोई साझा-परिवर्तनशील-स्थिति रेस कंडीशन नहीं

है, क्योंकि हर कॉल आमतौर पर एक ताज़ा, गैर-साझा ऑब्जेक्ट बनाती है। जिस एक जगह concurrency प्रासंगिक हो जाती है वह है अगर एक `ConcreteCreator` खुद कॉल्स में साझा स्थिति रखता और बदलता है (जैसे एक आंतरिक काउंटर या कैश) — उस बिंदु पर concurrency तर्क असल में उस खास साझा स्थिति के बारे में है, Factory Method पैटर्न के बारे में नहीं। सही ढंग से यह कहना कि "यहाँ कोई सार्थक concurrency कहानी नहीं है, और यहाँ ठीक-ठीक बताया गया है क्यों" एक गढ़ी हुई कहानी से बेहतर जवाब है।

✓ **Before you move on:** (1) Which single SOLID principle is this pattern's strongest, most direct payoff? (2) Under what specific circumstance does concurrency become relevant here at all, despite the pattern having no concurrency story by default?

✓ **आगे बढ़ने से पहले:** (1) कौन-सा अकेला SOLID सिद्धांत इस पैटर्न का सबसे मज़बूत, सबसे सीधा फ़ायदा है? (2) डिफ़ॉल्ट रूप से इस पैटर्न की कोई concurrency कहानी न होने के बावजूद, किस खास परिस्थिति में यहाँ concurrency बिल्कुल भी प्रासंगिक हो जाती है?

Part 16 — Advantages, Disadvantages & Trade-offs

Dimension	Advantage	Disadvantage / trade-off
Performance	Negligible overhead (one virtual call)	None meaningful — not a performance-motivated pattern either way
Scalability	New types added without touching existing tested code, at any team size	None inherent — scales cleanly with codebase growth
Maintainability	Creation logic and type-specific behavior live together, in one place per type	Can add real, unjustified ceremony if applied where only one type will ever exist
Readability	Client code reads cleanly against the <code>Product</code> interface, with no branching	An unfamiliar reader has to trace through the class hierarchy to see which concrete type gets created for a given path — less immediately obvious than a single switch statement
Security	Neutral	Neutral
Testing	Each <code>ConcreteCreator</code> is independently, cleanly testable	A Simple Factory's single method is often <i>more</i> directly testable with a simple table-driven test — worth naming as a genuine testing-simplicity trade-off in the other direction
Extensibility	The pattern's core strength — new types via new subclasses only	—

हिंदी सार: परफ़ॉर्मेंस — नगण्य ओवरहेड, कोई सार्थक नुकसान नहीं। स्केलेबिलिटी — मौजूदा टेस्ट किए गए कोड को छुए बिना नई टाइप्स जुड़ती हैं, कोई अंतर्निहित नुकसान नहीं। मेंटेनेबिलिटी — क्रिएशन लॉजिक और टाइप-विशिष्ट व्यवहार एक साथ रहते हैं, लेकिन अगर सिर्फ़ एक टाइप कभी अस्तित्व में रहेगी तो यह असली, अनुचित औपचारिकता जोड़ सकता है। पठनीयता — क्लाइंट कोड `Product` इंटरफ़ेस के खिलाफ़ साफ़ पढ़ता है, लेकिन एक अपरिचित पाठक को यह देखने के लिए क्लास पदानुक्रम में जाना पड़ता है कि किसी दिए गए पथ के

लिए कौन-सी कॉन्क्रीट टाइप बनती है। सुरक्षा — तटस्थ दोनों तरफ़। टेस्टिंग — हर ConcreteCreator स्वतंत्र रूप से टेस्ट करने योग्य है, लेकिन एक Simple Factory की अकेली मेथड अक्सर एक सादे टेबल-आधारित टेस्ट से ज़्यादा सीधे टेस्ट करने योग्य होती है। एक्सटेंसिबिलिटी — पैटर्न की मुख्य ताकत, सिर्फ़ नई सबक्लासेज़ के ज़रिए नई टाइप्स।

✓ **Before you move on:** (1) Name the one dimension where this pattern is close to a pure win with essentially no real downside. (2) Name the dimension where a Simple Factory can genuinely be *easier* to test than full Factory Method.

✓ **आगे बढ़ने से पहले:** (1) वह एक आयाम बताएँ जहाँ यह पैटर्न बिना किसी असली नुकसान के लगभग पूरी तरह फ़ायदेमंद है। (2) वह आयाम बताएँ जहाँ एक Simple Factory सचमुच पूरी Factory Method से आसान टेस्ट हो सकती है।

Part 17 — Pattern Comparisons

	Factory Method	Abstract Factory	Builder	Simple Factory (not GoF)
What varies	Which concrete class of ONE product type	Which concrete FAMILY of multiple related product types	HOW a complex object is assembled, step by step	Which concrete class, via one centralized method
Mechanism	Subclassing, one factory method per ConcreteCreator	Subclassing/ composition, one interface bundling several factory methods	Step-by-step builder object, often with fluent chaining	A static/free function with conditional logic, no subclassing
Extensibility mechanism	Add a new ConcreteCreator subclass	Add a new concrete factory implementing the whole family	Add new builder steps/variants	Edit the existing method's conditional
Best for	One product type, genuine ongoing subclass-level variation	Matched families that must stay visually/behaviorally consistent	Complex objects with many optional, combinable parameters	Simple, centralized, low-churn type selection
GoF pattern?	Yes	Yes	Yes	No — common pragmatic idiom, frequently mislabeled as "Factory"

हिंदी सार (तुलना टेबल): Factory Method में सिर्फ़ एक प्रोडक्ट टाइप की कॉन्क्रीट क्लास बदलती है, सबक्लासिंग के ज़रिए, नई ConcreteCreator सबक्लास जोड़कर एक्सटेंड होता है। Abstract Factory में कई संबंधित प्रोडक्ट टाइप्स का पूरा परिवार बदलता है, एक इंटरफ़ेस में कई factory मेथड्स बंधी होती हैं। Builder में यह बदलता है कि एक जटिल ऑब्जेक्ट चरण-दर-चरण कैसे

असेंबल होता है। Simple Factory (GoF पैटर्न नहीं) में एक केंद्रीकृत मेथड के ज़रिए कॉन्क्रीट क्लास बदलती है, कोई सबक्लासिंग नहीं, मौजूदा मेथड की शर्त एडिट करके एक्सटेंड होता है।

Decision table:

Situation	Reach for
One product type, real subclass-level extensibility need, ongoing	Factory Method
A <i>family</i> of related objects must be created consistently together	Abstract Factory
One complex object, many optional/combinable construction parameters	Builder
One product type, simple/rare changes, no real extensibility need	Simple Factory (and say so plainly — it's not a lesser answer)
Need a cheap copy of an already-configured instance, not a "which class" decision	Prototype

निर्णय टेबल — हिंदी सार: एक प्रोडक्ट टाइप, असली चल रही सबक्लास-स्तर एक्सटेंसिबिलिटी ज़रूरत → Factory Method। संबंधित ऑब्जेक्ट्स का एक परिवार लगातार एक साथ बनाया जाना चाहिए → Abstract Factory। एक जटिल ऑब्जेक्ट, कई वैकल्पिक/संयोजनीय निर्माण पैरामीटर्स → Builder। एक प्रोडक्ट टाइप, सरल/दुर्लभ बदलाव, कोई असली एक्सटेंसिबिलिटी ज़रूरत नहीं → Simple Factory (और यह खुलकर कहें — यह कमज़ोर जवाब नहीं है)। एक पहले से कॉन्फ़िगर्ड इंस्टेंस की सस्ती कॉपी चाहिए, "कौन-सी क्लास" वाला फैसला नहीं → Prototype।

✓ **Before you move on:** (1) What's the one-sentence distinction between Factory Method and Abstract Factory? (2) Why is "Simple Factory" not technically a GoF pattern, and why is that a fine, correct thing to say in an interview rather than something to hide?

✓ **आगे बढ़ने से पहले:** (1) Factory Method और Abstract Factory के बीच एक-वाक्य का भेद क्या है? (2) "Simple Factory" तकनीकी रूप से GoF पैटर्न क्यों नहीं है, और यह इंटरव्यू में कहने लायक एक सही बात क्यों है, छिपाने लायक चीज़ नहीं?

Part 18 — Production Bugs, AI-Generated Code Review & Testing

The flagship bug — `self` vs. `static` in an inherited factory method. A base `Creator`'s factory method uses `return new self();` instead of `return new static();`. Every subclass that *doesn't* override the factory method itself, but relies on inheriting it, silently gets an instance of the *base* class instead of the expected subclass — a correctness bug that often only surfaces later, as a confusing `instanceof` check failing somewhere downstream. Debug by checking `get_class()` on the returned object against the expected `ConcreteCreator`'s type at the point of construction.

मुख्य बग — एक इनहेरिटेड factory मेथड में `self` बनाम `static`। एक बेस `Creator` की factory मेथड `return new static();` के बजाय `return new self();` इस्तेमाल करती है। हर सबक्लास जो factory मेथड को खुद ओवरराइड नहीं

करती, बल्कि इसे इनहेरिट करने पर निर्भर करती है, चुपचाप अपेक्षित सबक्लास के बजाय बेस क्लास का एक इंस्टेंस पाती है — एक सटीकता बग जो अक्सर बाद में ही सामने आती है, आगे कहीं एक भ्रामक `instanceof` जाँच विफल होने के तौर पर। निर्माण के बिंदु पर लौटाए गए ऑब्जेक्ट पर `get_class()` की जाँच अपेक्षित `ConcreteCreator` की टाइप से करके डीबग करें।

LSP-violating `ConcreteProduct` . A new concrete product technically implements the `Product` interface's method signatures but violates behavioral expectations client code silently relies on (e.g., a `send()` method that doesn't actually throw on failure the way every other implementation does) — passes every type-check, breaks real behavior. This is a Liskov Substitution problem wearing a Factory Method costume; the fix is a stricter interface contract (documented invariants, or an interface-level test suite every `ConcreteProduct` must pass), not a Factory Method fix per se.

LSP का उल्लंघन करने वाला `ConcreteProduct` । एक नया कॉन्क्रीट प्रोडक्ट तकनीकी रूप से `Product` इंटरफ़ेस के मेथड सिग्नेचर्स इम्प्लीमेंट करता है लेकिन उन व्यवहारगत अपेक्षाओं का उल्लंघन करता है जिन पर क्लाइंट कोड चुपचाप निर्भर करता है (जैसे एक `send()` मेथड जो असल में विफलता पर वैसे श्रो नहीं करती जैसे हर दूसरा इम्प्लीमेंटेशन करता है) — हर टाइप-जाँच पास करती है, असली व्यवहार तोड़ती है। यह Factory Method का लबादा पहने एक Liskov Substitution समस्या है; फ़िक्स एक सख्त इंटरफ़ेस कॉन्ट्रैक्ट है (दस्तावेज़ीकृत अपरिवर्तनीय, या एक इंटरफ़ेस-स्तर टेस्ट सूट जो हर `ConcreteProduct` को पास करना चाहिए), कोई Factory Method फ़िक्स नहीं।

How AI coding assistants typically get this pattern wrong: - **Most common failure:** when asked to "add a factory for X," AI assistants very reliably produce a **Simple Factory** (one method, one switch) rather than genuine subclass-based Factory Method — which is often the *right* call, but the assistant rarely states that trade-off explicitly, leaving a reviewer to notice (or miss) that what was delivered isn't actually the GoF pattern the ticket or comment named. -

Second most common failure: AI-generated PHP factory methods inherited across a class hierarchy frequently default to `new self()` rather than `new static()`, reproducing the exact late-static-binding bug from this section's flagship bug — because the surface syntax looks identical and the failure is silent, not a parse or type error. - **What a reviewer should check**

before merging: (1) if the ticket explicitly asked for "Factory Method," does the delivered code actually use subclassing/polymorphic override, or is it a Simple Factory mislabeled — and if it's a Simple Factory, is that actually fine given the real requirement; (2) in PHP, does every `new self()` inside an inheritable factory method actually need to be `new static()` instead; (3) does every `ConcreteProduct` genuinely satisfy the `Product` interface's *behavioral* contract, not just its method signatures.

AI कोडिंग असिस्टेंट्स आमतौर पर इस पैटर्न को कैसे गलत करते हैं: - **सबसे आम विफलता:** जब "X के लिए एक factory जोड़ो" कहा जाता है, AI असिस्टेंट्स बहुत भरोसेमंद तरीके से एक **Simple Factory** (एक मेथड, एक switch) बनाते हैं, असली सबक्लास-आधारित Factory Method नहीं — जो अक्सर सही फ़ैसला होता है, लेकिन असिस्टेंट शायद ही कभी उस ट्रेड-ऑफ़ को खुलकर बताता है, एक रिव्यूअर को यह नोटिस करने (या चूकने) के लिए छोड़ते हुए कि जो डिलीवर हुआ वह असल में वह GoF पैटर्न नहीं है जिसका टिकट या कमेंट ने नाम लिया था। - **दूसरी सबसे आम विफलता:** AI-जनित PHP factory मेथड्स, जो एक क्लास पदानुक्रम में इनहेरिट होती हैं, अक्सर डिफ़ॉल्ट रूप से `new static()` के बजाय `new self()` इस्तेमाल करती हैं, इस सेक्शन की मुख्य बग वाली ठीक वही late-static-binding बग दोहराते हुए — क्योंकि सतही सिंटैक्स एक जैसा दिखता है और विफलता चुप है, कोई पार्स या टाइप एरर नहीं। - **मर्ज करने से पहले एक रिव्यूअर को क्या जाँचना चाहिए:** (1) अगर टिकट ने खुलकर "Factory Method" माँगा है, तो क्या डिलीवर किया गया कोड असल में सबक्लासिंग/पॉलीमॉर्फिक ओवरराइड इस्तेमाल करता है, या यह एक

गलत-लेबल की गई Simple Factory है — और अगर यह Simple Factory है, तो क्या असली ज़रूरत को देखते हुए यह असल में ठीक है; (2) PHP में, क्या एक इनहेरिट-करने-योग्य factory मेथड के अंदर हर `new self()` को असल में `new static()` होना चाहिए; (3) क्या हर `ConcreteProduct` सचमुच `Product` इंटरफ़ेस के व्यवहारगत कॉन्ट्रैक्ट को संतुष्ट करता है, सिर्फ़ उसके मेथड सिग्नेचर्स को नहीं।

Testing strategy:

```
public function test_each_concrete_creator_returns_the_correct_product_type(): void
{
    $this->assertInstanceOf(EmailNotification::class, (new EmailNotificationCreator())->createNotification());
    $this->assertInstanceOf(SmsNotification::class, (new SmsNotificationCreator())->createNotification());
}

public function test_late_static_binding_returns_the_calling_subclass_not_the_base(): void
{
    // Guards specifically against the `self` vs `static` regression.
    $result = ConcreteCreatorSubclass::create();
    $this->assertInstanceOf(ConcreteCreatorSubclass::class, $result);
    $this->assertNotSame(BaseCreator::class, get_class($result));
}
```

The second test exists specifically because the `self / static` bug produces a *type-correct-looking* object (it's still a valid PHP object, code doesn't crash) — only an identity/class check catches it, the same lesson as Prototype's and Singleton's identity-testing rules, applied to a different failure mode.

दूसरा टेस्ट खास तौर पर इसलिए मौजूद है क्योंकि `self / static` बग एक टाइप-सही-दिखने-वाला ऑब्जेक्ट पैदा करती है (यह अब भी एक वैध PHP ऑब्जेक्ट है, कोड क्रैश नहीं होता) — सिर्फ़ एक पहचान/क्लास जाँच ही इसे पकड़ती है, वही सबक जो Prototype और Singleton के पहचान-टेस्टिंग नियमों का है, एक अलग विफलता ढंग पर लागू।

Code review checklist: every inheritable factory method in PHP uses `new static()`, not `new self()`, unless deliberately pinning to the base class; every `ConcreteProduct` is covered by a shared behavioral contract test, not just a type-check; if the ticket says "Factory Method" but the diff shows a single switch-based method, that's flagged and discussed explicitly rather than silently merged either way.

कोड रिव्यू चेकलिस्ट: PHP में हर इनहेरिट-करने-योग्य factory मेथड `new static()` इस्तेमाल करती है, `new self()` नहीं, जब तक जानबूझकर बेस क्लास से नहीं बाँधा गया हो; हर `ConcreteProduct` एक साझा व्यवहारगत कॉन्ट्रैक्ट टेस्ट से कवर है, सिर्फ़ एक टाइप-जाँच से नहीं; अगर टिकट "Factory Method" कहता है लेकिन diff एक अकेली switch-आधारित मेथड दिखाता है, तो इसे किसी भी तरह चुपचाप मर्ज करने के बजाय खुलकर चिह्नित और चर्चा की जाती है।

✓ **Before you move on:** (1) What's the specific PHP bug this pattern is most associated with, and why does it fail silently? (2) Why does an `instanceof`-only test suite fail to catch the `self / static` bug?

✓ **आगे बढ़ने से पहले:** (1) इस पैटर्न से सबसे ज़्यादा जुड़ी खास PHP बग क्या है, और यह चुपचाप क्यों विफल होती है? (2) सिर्फ़ `instanceof` वाला टेस्ट सूट `self / static` बग पकड़ने में क्यों विफल रहता है?

Part 19 — Refactoring Journey

Full code for every stage lives in `Factory.php`; this narrates the reasoning connecting each one.

हर चरण का पूरा कोड `Factory.php` में है; यह हर एक को जोड़ने वाला तर्क बताता है।

Stage 1 — Terrible (*where most engineers start, no shame in it*): the same type-branching `if / elseif` block, copy-pasted across every place a notification gets created.

चरण 1 — भयानक (जहाँ से ज्यादातर इंजीनियर शुरू करते हैं, इसमें कोई शर्म नहीं): वही टाइप-शाखाबद्ध `if / elseif` ब्लॉक, हर उस जगह कॉपी-पेस्ट जहाँ एक नोटिफिकेशन बनती है।

Stage 2 — Bad, but a realistic first instinct (*often written by a mid-level engineer under time pressure*): a single centralized Simple Factory method — genuinely better than Stage 1, and honestly *sufficient* for many real systems. Listed as "bad" here only relative to the specific requirement this journey is walking through (per-channel behavioral variation, not just creation) — worth stating clearly that Stage 2 is often the *correct* stopping point for a different, simpler requirement.

चरण 2 — बुरा, मगर एक यथार्थवादी पहला अंतर्ज्ञान (अक्सर समय के दबाव में एक मिड-लेवल इंजीनियर द्वारा लिखा गया): एक अकेली केंद्रीकृत Simple Factory मेथड — चरण 1 से सचमुच बेहतर, और ईमानदारी से कई असली सिस्टम्स के लिए पर्याप्त। यहाँ "बुरा" सिर्फ़ इस यात्रा की खास ज़रूरत (प्रति-चैनल व्यवहारगत बदलाव, सिर्फ़ क्रिएशन नहीं) के सापेक्ष चिह्नित है — स्पष्ट रूप से कहने लायक कि चरण 2 अक्सर एक अलग, सरल ज़रूरत के लिए सही रुकने का बिंदु होता है।

Stage 3 — Average, and the most dangerous stage in the whole journey (*a senior engineer moving fast, or code that later drifts*): a `Creator` hierarchy is introduced, but the base factory method uses `new self()` instead of `new static()`, so subclasses that rely on inheriting rather than overriding the factory method silently get the wrong type. Passes a casual smoke test (it *does* return *some* valid `Product`), looks finished, and is wrong in a way that only an identity check catches — exactly Part 18's flagship bug.

चरण 3 — औसत, और पूरी यात्रा का सबसे खतरनाक चरण (एक सीनियर इंजीनियर तेज़ी से आगे बढ़ रहा है, या कोड जो बाद में भटक जाता है): एक `Creator` पदानुक्रम पेश किया जाता है, लेकिन बेस factory मेथड `new static()` के बजाय `new self()` इस्तेमाल करती है, इसलिए वे सबक्लासेज़ जो factory मेथड को ओवरराइड करने के बजाय इनहेरिट करने पर निर्भर करती हैं, चुपचाप ग़लत टाइप पाती हैं। एक साधारण स्मोक टेस्ट पास कर जाता है (यह कोई वैध `Product` लौटाता तो है), पूरा दिखता है, और एक ऐसे तरीके से ग़लत है जिसे सिर्फ़ एक पहचान जाँच पकड़ती है — ठीक Part 18 की मुख्य बग।

Stage 4 — Pattern correctly applied (*what a rigorous senior/staff engineer ships*): fixes the `static` binding, adds the identity-check test proving it, and ensures every `ConcreteProduct` is covered by a shared behavioral-contract test, not just a type-check.

चरण 4 — पैटर्न सही ढंग से लगाया गया (जो एक सख्त सीनियर/स्टाफ़ इंजीनियर शिप करता है): `static` बाइंडिंग ठीक करता है, इसे साबित करने वाला पहचान-जाँच टेस्ट जोड़ता है, और सुनिश्चित करता है कि हर `ConcreteProduct` एक साझा व्यवहारगत-कॉन्ट्रैक्ट टेस्ट से कवर हो, सिर्फ़ एक टाइप-जाँच से नहीं।

Stage 5 — Production-ready (*staff-level judgment about the surrounding system, not just the class*): documents the extensibility decision in an ADR (Part 12), instruments creation with metrics if creation cost or type-distribution is operationally meaningful (rarely the case for this

pattern, but worth checking rather than assuming), and — critically — periodically revisits whether the requirement that originally justified full Factory Method over a Simple Factory still holds, since over-time requirement drift is exactly what can make yesterday's correct abstraction into today's unnecessary ceremony.

चरण 5 — प्रोडक्शन-रेडी (आस-पास के सिस्टम के बारे में स्टाफ़-स्तर का निर्णय, सिर्फ़ क्लास के बारे में नहीं): एक ADR (Part 12) में एक्सटेंसिबिलिटी फ़ैसला दस्तावेज़ीकृत करता है, अगर क्रिएशन लागत या टाइप-वितरण परिचालन रूप से सार्थक है तो क्रिएशन को मेट्रिक्स से इंस्ट्रुमेंट करता है (इस पैटर्न के लिए शायद ही कभी ऐसा हो, लेकिन मान लेने के बजाय जाँचना ज़रूरी), और — महत्वपूर्ण रूप से — समय-समय पर यह देखता है कि जिस ज़रूरत ने मूल रूप से Simple Factory के मुकाबले पूरी Factory Method को जायज़ ठहराया था वह अब भी मौजूद है या नहीं, क्योंकि समय के साथ ज़रूरत का बदलना ही वह चीज़ है जो कल के सही अमूर्तन को आज की अनावश्यक औपचारिकता बना सकती है।

✓ **Before you move on:** (1) Why is Stage 2 (Simple Factory) not actually "wrong" in an absolute sense, only relative to this specific journey's requirement? (2) What specifically makes Stage 3 the most dangerous stage to leave in production?

✓ **आगे बढ़ने से पहले:** (1) चरण 2 (Simple Factory) पूर्ण अर्थ में असल में "ग़लत" क्यों नहीं है, सिर्फ़ इस खास यात्रा की ज़रूरत के सापेक्ष क्यों है? (2) ठीक-ठीक क्या चीज़ चरण 3 को प्रोडक्शन में छोड़ने के लिए सबसे ख़तरनाक चरण बनाती है?

Part 20 — Practices, Mistakes & Traps

Junior mistakes: reaching for a full `Creator / ConcreteCreator` class hierarchy for a type that will only ever have one variant; not realizing a Simple Factory is a legitimate, sufficient answer for many real requirements.

शुरुआती ग़लतियाँ: एक ऐसी टाइप के लिए पूरी `Creator / ConcreteCreator` क्लास पदानुक्रम की ओर पहुँचना जिसका हमेशा सिर्फ़ एक ही वेरिएंट रहेगा; यह न समझना कि कई असली ज़रूरतों के लिए एक Simple Factory एक वैध, पर्याप्त जवाब है।

Mid-level mistakes: conflating Factory Method with Abstract Factory when the actual requirement is a *family* of related objects, not one; in PHP, using `new self()` in an inheritable factory method without realizing the late-static-binding implication.

मिड-लेवल ग़लतियाँ: जब असली ज़रूरत संबंधित ऑब्जेक्ट्स का एक परिवार हो, एक नहीं, तब Factory Method को Abstract Factory के साथ गड़मड़ करना; PHP में, एक इनहेरिट-करने-योग्य factory मेथड में `new self()` इस्तेमाल करना बिना late-static-binding के प्रभाव को समझे।

Senior mistakes: applying full Factory Method preemptively for "future extensibility" that never materializes, adding real ongoing maintenance cost for a payoff that's never collected — the inverse failure mode from the junior mistake, and arguably more common among engineers who've internalized "always use the proper pattern" too rigidly; failing to revisit an earlier Factory Method decision when the original justification (genuine subclass-level behavioral variation) has quietly stopped being true.

सीनियर ग़लतियाँ: "भविष्य की एक्सटेंसिबिलिटी" के लिए पहले से ही पूरी Factory Method लगाना जो कभी साकार नहीं होती, एक ऐसे फ़ायदे के लिए असली चल रही रखरखाव लागत जोड़ना जो कभी वसूल नहीं होता — शुरुआती ग़लती से उलट विफलता ढंग, और संभवतः उन इंजीनियरों में ज़्यादा आम जिन्होंने "हमेशा सही पैटर्न इस्तेमाल करो" को बहुत सख्ती से अपना लिया है; एक पहले के

Factory Method फ़ैसले को दोबारा न देखना जब मूल औचित्य (असली सबक्लास-स्तर व्यवहारगत बदलाव) चुपचाप सच होना बंद हो चुका हो।

Interview follow-up questions that catch memorized-but-shallow understanding:

"You said Factory Method — walk me through why a Simple Factory wouldn't have been sufficient here." (catches candidates who reach for the "proper" GoF pattern reflexively without a real justification.) - "If I ask for a family of three related objects that must stay visually consistent, does your design still work?" (catches the Factory-Method/Abstract-Factory conflation.) - "In PHP, what happens if this base class's factory method uses `new self()` and a subclass doesn't override it?" (catches candidates who can implement the pattern's shape without understanding this specific, real gotcha.) - "What's the concurrency story for this pattern?" (the correct, confident answer is usually 'there isn't one, and here's specifically why' — candidates who manufacture a concurrency concern here are pattern-matching from Singleton without actually reasoning about this pattern's mechanics.)

इंटरव्यू फ़ॉलो-अप सवाल जो रटी-मगर-सतही समझ पकड़ते हैं: - "आपने Factory Method कहा — मुझे बताइए कि यहाँ एक Simple Factory पर्याप्त क्यों नहीं होती।" (उन उम्मीदवारों को पकड़ता है जो बिना असली औचित्य के बिना सोचे "सही" GoF पैटर्न की ओर पहुँचते हैं।) - "अगर मैं तीन संबंधित ऑब्जेक्ट्स के एक परिवार की माँग करूँ जिन्हें दिखने में लगातार मेल खाना चाहिए, तो क्या आपका डिज़ाइन अब भी काम करता है?" (Factory-Method/Abstract-Factory गड़बड़ को पकड़ता है।) - "PHP में, अगर इस बेस क्लास की factory मेथड `new self()` इस्तेमाल करती है और एक सबक्लास इसे ओवरराइड नहीं करती, तो क्या होता है?" (उन उम्मीदवारों को पकड़ता है जो इस खास, असली गड़बड़ी को समझे बिना पैटर्न का आकार इम्प्लीमेंट कर सकते हैं।) - "इस पैटर्न के लिए concurrency कहानी क्या है?" (सही, भरोसेमंद जवाब आमतौर पर 'कोई नहीं है, और यहाँ ठीक-ठीक बताया गया है क्यों' होता है — जो उम्मीदवार यहाँ एक concurrency चिंता गढ़ते हैं वे इस पैटर्न के तंत्र के बारे में असल में तर्क किए बिना Singleton से पैटर्न-मैच कर रहे होते हैं।)

✓ **Before you move on:** (1) What's the mid-level mistake and the senior mistake with this pattern, and how are they nearly opposite failure modes? (2) Which follow-up question specifically tests whether a candidate is pattern-matching from Singleton rather than reasoning about Factory Method on its own terms?

✓ **आगे बढ़ने से पहले:** (1) इस पैटर्न के साथ मिड-लेवल गलती और सीनियर गलती क्या हैं, और वे लगभग विपरीत विफलता ढंग कैसे हैं? (2) कौन-सा फ़ॉलो-अप सवाल खास तौर पर यह जाँचता है कि क्या एक उम्मीदवार Factory Method के बारे में अपने आप तर्क करने के बजाय Singleton से पैटर्न-मैच कर रहा है?

Part 21 — Interview Question Bank & Coding Problems

Curated, high-signal, roughly 7 per level. Total questions delivered: 35.

चुनी हुई, उच्च-संकेत, प्रति स्तर लगभग 7। कुल दी गई सवालों की संख्या: 35।

Beginner (7)

1. *What problem does Factory Method solve?* — Wrong: "it makes objects faster." — Good: "lets a class create objects without hard-coding the concrete class." — Excellent: adds

"...specifically by deferring that decision to a subclass, via an overridden method." —

Follow-up: "give a real example."

हिंदी: Factory Method कौन-सी समस्या हल करता है? — ग़लत: "यह ऑब्जेक्ट्स को तेज़ बनाता है।" — अच्छा: "एक क्लास को कॉन्क्रीट क्लास हार्ड-कोड किए बिना ऑब्जेक्ट्स बनाने देता है।" — उत्कृष्ट: जोड़ता है "...खास तौर पर उस फ़ैसले को एक ओवरराइड की गई मेथड के ज़रिए एक सबक्लास तक टालकर।" — फ़ॉलो-अप: "एक असली उदाहरण दीजिए।"

1. *What's the difference between Factory Method and just calling `new`?* — Good: "calling `new` directly hard-codes the concrete class; Factory Method lets that vary." — Excellent: names the OCP payoff specifically. — Follow-up: "when would calling `new` directly still be the right choice?"

हिंदी: Factory Method और सीधे `new` कॉल करने में क्या अंतर है? — अच्छा: "सीधे `new` कॉल करना कॉन्क्रीट क्लास को हार्ड-कोड करता है; Factory Method इसे बदलने देता है।" — उत्कृष्ट: खास तौर पर OCP फ़ायदे का नाम लेता है। — फ़ॉलो-अप: "सीधे `new` कॉल करना कब भी सही चुनाव होगा?"

1. *What roles/classes does this pattern involve?* — Good: names Creator and ConcreteCreator. — Excellent: also names Product and ConcreteProduct, and states the relationship between all four. — Follow-up: "which of these is abstract, and which is concrete?"

हिंदी: इस पैटर्न में कौन-सी भूमिकाएँ/क्लासेज़ शामिल हैं? — अच्छा: Creator और ConcreteCreator का नाम लेता है। — उत्कृष्ट: Product और ConcreteProduct का भी नाम लेता है, और चारों के बीच संबंध बताता है। — फ़ॉलो-अप: "इनमें से कौन-सी ऐब्स्ट्रेक्ट है, और कौन-सी कॉन्क्रीट?"

1. *Is Factory Method structural, creational, or behavioral?* — Good: "creational." — Excellent: explains why (it's about controlling object instantiation, not composing objects or defining communication between them). — Follow-up: "name two other creational patterns."

हिंदी: क्या Factory Method स्ट्रक्चरल, क्रिएशनल, या बिहेवियरल है? — अच्छा: "क्रिएशनल।" — उत्कृष्ट: बताता है क्यों (यह ऑब्जेक्ट इंस्टैंशिएशन को नियंत्रित करने के बारे में है, ऑब्जेक्ट्स को कंपोज़ करने या उनके बीच संचार परिभाषित करने के बारे में नहीं)। — फ़ॉलो-अप: "दो और क्रिएशनल पैटर्न्स के नाम बताइए।"

1. *What is a "Simple Factory," and is it the same as Factory Method?* — Wrong: "yes, same thing." — Good: "no — Simple Factory is one method with a switch, no subclassing." — Excellent: adds that Simple Factory isn't formally a GoF pattern at all, just a common pragmatic idiom. — Follow-up: "when is Simple Factory actually the better choice?"

हिंदी: "Simple Factory" क्या है, और क्या यह Factory Method जैसी ही है? — ग़लत: "हाँ, एक ही चीज़।" — अच्छा: "नहीं — Simple Factory एक switch वाली एक मेथड है, कोई सबक्लासिंग नहीं।" — उत्कृष्ट: जोड़ता है कि Simple Factory औपचारिक रूप से बिल्कुल भी GoF पैटर्न नहीं है, बस एक आम व्यावहारिक तरीक़ा है। — फ़ॉलो-अप: "Simple Factory असल में कब बेहतर चुनाव है?"

1. *Give a real-world (non-code) example of Factory Method.* — Good: any reasonable analogy. — Excellent: an analogy that correctly captures BOTH "shared process" and "varying creation decision" (e.g., the pizza-franchise analogy). — Follow-up: "where does that analogy break down?"

हिंदी: Factory Method का एक असली-दुनिया (गैर-कोड) उदाहरण दीजिए। — अच्छा: कोई भी उचित उपमा। — उत्कृष्ट: एक उपमा जो "साझा प्रक्रिया" और "बदलता क्रिएशन फ़ैसला" दोनों को सही ढंग से पकड़ती है (जैसे, पिज़्ज़ा-फ़्रैंचाइज़ी उपमा)। — फ़ॉलो-अप: "वह उपमा कहाँ टूटती है?"

1. *Does the client code need to know the concrete product class?* — Good: "no, it only knows the Product interface." — Excellent: explains why that's the whole point — it's what makes adding new types safe. — Follow-up: "what would break if client code DID depend on the concrete class?"

हिंदी: क्या क्लाइंट कोड को कॉन्क्रीट प्रोडक्ट क्लास जानने की ज़रूरत है? — अच्छा: "नहीं, यह सिर्फ़ Product इंटरफ़ेस जानता है।" — उत्कृष्ट: बताता है कि यही पूरा मक़सद है — यही नई टाइप्स जोड़ने को सुरक्षित बनाता है। — फ़ॉलो-अप: "अगर क्लाइंट कोड कॉन्क्रीट क्लास पर निर्भर करता, तो क्या टूटता?"

Intermediate (7)

1. *Implement a small Factory Method hierarchy for two or three related types.* — Wrong: implements a Simple Factory and calls it Factory Method without noting the difference. — Good: correct subclass-based implementation. — Excellent: also states explicitly why subclassing was chosen over a Simple Factory for this specific case. — Follow-up: "add a third type — what changes?"

हिंदी: दो या तीन संबंधित टाइप्स के लिए एक छोटी Factory Method पदानुक्रम इम्प्लीमेंट करें। — ग़लत: एक Simple Factory इम्प्लीमेंट करता है और भेद बताए बिना उसे Factory Method कहता है। — अच्छा: सही सबक्लास-आधारित इम्प्लीमेंटेशन। — उत्कृष्ट: यह भी खुलकर बताता है कि इस ख़ास मामले के लिए Simple Factory के मुक़ाबले सबक्लासिंग क्यों चुनी गई। — फ़ॉलो-अप: "एक तीसरी टाइप जोड़ें — क्या बदलता है?"

1. *What's the difference between Factory Method and Abstract Factory?* — Good: "Abstract Factory creates families of related objects." — Excellent: gives a concrete example needing a family (matched UI widget set) versus one needing just Factory Method (one receipt type varying by provider). — Follow-up: "when would you need both together?"

हिंदी: Factory Method और Abstract Factory में क्या अंतर है? — अच्छा: "Abstract Factory संबंधित ऑब्जेक्ट्स के परिवार बनाता है।" — उत्कृष्ट: एक परिवार की ज़रूरत वाला ठोस उदाहरण देता है (मेल खाता UI विजेट सेट) बनाम सिर्फ़ Factory Method की ज़रूरत वाला (प्रोवाइडर से बदलने वाली एक रसीद टाइप)। — फ़ॉलो-अप: "आपको दोनों एक साथ कब चाहिए होंगे?"

1. *In PHP, what's the difference between `new self()` and `new static()` inside a factory method?* — Wrong: "they're the same." — Good: "`static` respects late static binding, `self` doesn't." — Excellent: explains the concrete failure mode when a subclass inherits rather than overrides the method. — Follow-up: "write a test that would catch this bug."

हिंदी: PHP में, एक factory मेथड के अंदर `new self()` और `new static()` में क्या अंतर है? — ग़लत: "वे एक जैसे हैं।" — अच्छा: "`static` late static binding का सम्मान करता है, `self` नहीं।" — उत्कृष्ट: जब एक सबक्लास मेथड को ओवरराइड करने के बजाय इनहेरिट करती है तो ठोस विफलता ढंग समझाता है। — फ़ॉलो-अप: "एक टेस्ट लिखिए जो इस बग को पकड़े।"

1. *When would you choose Builder over Factory Method?* — Good: "when construction involves many optional steps, not just picking a class." — Excellent: gives a concrete example (a `SubscriptionPlan` with combinable add-ons) and explains why Factory Method's single-decision shape doesn't fit that. — Follow-up: "could you combine both in one system?"

हिंदी: आप Factory Method के मुकाबले Builder कब चुनेंगे? — अच्छा: "जब निर्माण में कई वैकल्पिक चरण शामिल हों, सिर्फ एक क्लास चुनना नहीं।" — उत्कृष्ट: एक ठोस उदाहरण देता है (संयोजनीय ऐड-ऑन्स वाला एक `SubscriptionPlan`) और बताता है कि Factory Method का एक-फ़ैसला आकार उसमें फ़िट क्यों नहीं बैठता। — फ़ॉलो-अप: "क्या आप एक सिस्टम में दोनों को जोड़ सकते हैं?"

1. *Does Factory Method have a meaningful thread-safety story?* — Good: "generally no, since it usually produces fresh, independent objects." — Excellent: names the specific exception (a `ConcreteCreator` that itself holds and mutates shared state). — Follow-up: "contrast that with Singleton's concurrency story."

हिंदी: क्या Factory Method की कोई सार्थक थ्रेड-सेफ्टी कहानी है? — अच्छा: "आमतौर पर नहीं, क्योंकि यह आमतौर पर ताज़ा, स्वतंत्र ऑब्जेक्ट्स बनाता है।" — उत्कृष्ट: खास अपवाद का नाम लेता है (एक `ConcreteCreator` जो खुद साझा स्थिति रखता और बदलता है)। — फ़ॉलो-अप: "उसकी तुलना Singleton की concurrency कहानी से कीजिए।"

1. *How would you unit test a Factory Method hierarchy?* — Good: instance-type checks per `ConcreteCreator`. — Excellent: adds a shared behavioral-contract test every `ConcreteProduct` must pass, not just `instanceof` checks. — Follow-up: "why isn't `instanceof` alone sufficient?"

हिंदी: आप एक Factory Method पदानुक्रम को कैसे यूनिट-टेस्ट करेंगे? — अच्छा: हर `ConcreteCreator` के लिए इंस्टेंस-टाइप जाँचें। — उत्कृष्ट: एक साझा व्यवहारगत-कॉन्ट्रैक्ट टेस्ट जोड़ता है जो हर `ConcreteProduct` को पास करना चाहिए, सिर्फ `instanceof` जाँचें नहीं। — फ़ॉलो-अप: "सिर्फ `instanceof` पर्याप्त क्यों नहीं है?"

1. *Your team has a Simple Factory that's grown to 12 branches in one method. When, if ever, do you refactor to Factory Method?* — Good: "when it gets hard to maintain." — Excellent: names the specific trigger — when *behavior*, not just creation, starts needing to vary per type, or when the single method's blast radius becomes a real risk (Part 13's field note). — Follow-up: "what's the actual refactoring sequence you'd follow?"

हिंदी: आपकी टीम के पास एक Simple Factory है जो एक मेथड में 12 शाखाओं तक बढ़ गई है। अगर कभी, तो आप Factory Method में कब रीफैक्टर करेंगे? — अच्छा: "जब इसे बनाए रखना मुश्किल हो जाए।" — उत्कृष्ट: खास ट्रिगर का नाम लेता है — जब व्यवहार, सिर्फ क्रिएशन नहीं, प्रति-टाइप बदलने की ज़रूरत शुरू हो, या जब अकेली मेथड का ब्लास्ट-रेडियस एक असली जोखिम बन जाए (Part 13 का फ़ील्ड नोट)। — फ़ॉलो-अप: "आप असल में कौन-सा रीफैक्टरिंग क्रम अपनाएँगे?"

Senior (7)

1. *Design the notification-creator hierarchy from Part 5's mock drill, and justify every design decision as you go.* — Excellent answer states the OCP-driven justification unprompted, correctly distinguishes it from Simple Factory, and correctly identifies the "matched family" follow-up as an Abstract Factory shift rather than forcing Factory Method to cover it.

हिंदी: Part 5 की मॉक ड्रिल से नोटिफिकेशन-क्रिएटर पदानुक्रम डिज़ाइन करें, और आगे बढ़ते हुए हर डिज़ाइन फैसले को सही ठहराएँ। — उत्कृष्ट जवाब बिना पूछे OCP-प्रेरित औचित्य बताता है, इसे Simple Factory से सही ढंग से अलग करता है, और "मेल खाता परिवार" फ़ॉलो-अप को Factory Method को इसे कवर करने के लिए मजबूर करने के बजाय एक Abstract Factory बदलाव के तौर पर सही ढंग से पहचानता है।

1. *A codebase has adopted Factory Method for a type that has had exactly one implementation for two years, with no plans to add more. What's your recommendation?* —

Excellent answer: recommends simplifying back down to a plain constructor or Simple Factory, explicitly naming this as removing unjustified ceremony — the "senior mistake" from Part 20, caught and corrected.

हिंदी: एक कोडबेस ने एक ऐसी टाइप के लिए Factory Method अपनाया है जिसका दो साल से ठीक एक ही इम्प्लीमेंटेशन रहा है, और कोई और जोड़ने की योजना नहीं है। आपकी सिफ़ारिश क्या है? — उत्कृष्ट जवाब: वापस एक सादे कंस्ट्रक्टर या Simple Factory तक सरल करने की सिफ़ारिश करता है, खुलकर इसे अनुचित औपचारिकता हटाने के तौर पर नाम देता है — Part 20 की "सीनियर ग़लती," पकड़ी और सुधारी गई।

1. *Explain precisely what Laravel's `Manager` class family (`CacheManager` , `SessionManager` , etc.) is doing, and whether it's "really" Factory Method.* — Excellent answer describes the `createDriver()` → `create{Name}Driver()` naming-convention dispatch with per-instance caching, and correctly states this is closer to a Simple-Factory-plus-registry hybrid than textbook subclass-based Factory Method — testing the Part 11 distinction precisely, not just recognition that "Laravel uses factories somewhere."

हिंदी: ठीक-ठीक बताएँ कि Laravel का `Manager` क्लास परिवार (`CacheManager` , `SessionManager` , आदि) क्या कर रहा है, और क्या यह "सचमुच" Factory Method है। — उत्कृष्ट जवाब `createDriver()` → `create{Name}Driver()` नामकरण-परंपरा डिस्पैच को प्रति-इंस्टेंस कैशिंग के साथ बताता है, और सही ढंग से कहता है कि यह टेक्स्टबुक सबक्लास-आधारित Factory Method से ज़्यादा एक Simple-Factory-प्लस-रजिस्ट्री हाइब्रिड के करीब है — Part 11 के भेद को सटीक रूप से जाँचना, सिर्फ़ यह पहचान नहीं कि "Laravel कहीं factories इस्तेमाल करता है।"

1. *How would you migrate a Simple Factory to Factory Method without breaking existing callers?* — Excellent answer: introduce the `Creator / ConcreteCreator` hierarchy behind the existing Simple Factory's public interface first (an internal refactor), verify behavior parity with tests, then only later expose the new extensibility surface to callers who need it — an incremental, low-risk sequencing answer rather than a big-bang rewrite.

हिंदी: आप मौजूदा कॉलर्स को तोड़े बिना एक Simple Factory को Factory Method में कैसे माइग्रेट करेंगे? — उत्कृष्ट जवाब: पहले मौजूदा Simple Factory के सार्वजनिक इंटरफ़ेस के पीछे `Creator / ConcreteCreator` पदानुक्रम पेश करें (एक आंतरिक रीफ़ैक्टर), टेस्ट्स से व्यवहार समानता सत्यापित करें, फिर बाद में ही ज़रूरतमंद कॉलर्स को नई एक्सटेंसिबिलिटी सतह उजागर करें — एक बड़े-धमाके वाले फिर से लिखने के बजाय एक क्रमिक, कम-जोखिम वाला क्रम जवाब।

1. *A junior engineer submits a PR using `new self()` inside a base class's factory method, inherited (not overridden) by three subclasses. Review it.* — Excellent answer catches the late-static-binding bug specifically, explains the concrete failure (all three subclasses silently construct the base class), and requests the `static` fix plus an identity-check test — not just a vague "looks fine" or an overly harsh rewrite demand.

हिंदी: एक शुरुआती इंजीनियर एक बेस क्लास की factory मेथड के अंदर `new self()` इस्तेमाल करता हुआ एक PR सबमिट करता है, जो तीन सबक्लासेज़ द्वारा इनहेरिट (ओवरराइड नहीं) की गई है। इसे रिव्यू करें। — उत्कृष्ट जवाब खास तौर पर late-static-binding बग पकड़ता है, ठोस विफलता समझाता है (तीनों सबक्लासेज़ चुपचाप बेस क्लास बनाती हैं), और `static` फ़िक्स के साथ एक पहचान-जाँच टेस्ट की माँग करता है — सिर्फ़ एक अस्पष्ट "ठीक लगता है" या बहुत सख्त फिर से लिखने की माँग नहीं।

1. *When does Factory Method's OCP benefit stop being worth its structural cost?* — Excellent answer: when the rate of new-type additions is low enough that the "add a subclass, don't touch existing code" benefit is rarely exercised, while the ongoing cost (more files, more

indirection for readers) is paid on every read — a genuine, defensible engineering judgment call, not a rule.

हिंदी: Factory Method का OCP फ़ायदा अपनी संरचनात्मक लागत के लायक होना कब बंद हो जाता है? — उत्कृष्ट जवाब: जब नई-टाइप जुड़ने की दर इतनी कम हो कि "एक सबक्लास जोड़ो, मौजूदा कोड मत छुओ" फ़ायदा शायद ही कभी इस्तेमाल होता है, जबकि चल रही लागत (ज़्यादा फ़ाइलें, पाठकों के लिए ज़्यादा अप्रत्यक्षता) हर पढ़ाई पर चुकाई जाती है — एक असली, बचाव-योग्य इंजीनियरिंग निर्णय, कोई नियम नहीं।

1. *Compare Factory Method's extensibility mechanism to a plugin-registry pattern (e.g., registering handlers by string key in a container).* — Excellent answer: both achieve "add new types without modifying existing code," but a registry is data-driven (can be configured/extended at runtime, even without a new deploy in some architectures) while Factory Method's mechanism is compile-time/class-structure-driven (each new type is a new subclass, requiring a code change and deploy) — a real, nuanced trade-off worth naming.

हिंदी: Factory Method के एक्सटेंसिबिलिटी तंत्र की तुलना एक प्लगइन-रजिस्ट्री पैटर्न (जैसे, एक कंटेनर में स्ट्रिंग-की से हैंडलर्स रजिस्टर करना) से कीजिए। — उत्कृष्ट जवाब: दोनों "मौजूदा कोड बदले बिना नई टाइप्स जोड़ना" हासिल करते हैं, लेकिन एक रजिस्ट्री डेटा-प्रेरित है (कुछ आर्किटेक्चर्स में बिना नई डिप्लॉय के भी रनटाइम पर कॉन्फ़िगर/एक्सटेंड की जा सकती है) जबकि Factory Method का तंत्र कंपाइल-टाइम/क्लास-स्ट्रक्चर-प्रेरित है (हर नई टाइप एक नई सबक्लास है, जिसे कोड बदलाव और डिप्लॉय चाहिए) — नाम लेने लायक एक असली, सूक्ष्म ट्रेड-ऑफ़।

Staff/Principal (7)

1. *Your org's notification system has grown to 15 ConcreteCreator subclasses over three years, and onboarding engineers say the class hierarchy is hard to navigate. Diagnose and propose a fix.* — Excellent answer: recognizes this as a legitimate cost of the pattern at scale (many small files, no single place to see "all supported types" at a glance), and proposes a registry/index (a simple NotificationCreatorRegistry mapping type-keys to creator instances) layered on top of the existing Factory Method hierarchy — improving discoverability without abandoning the extensibility mechanism that's still earning its keep.

हिंदी: आपके संगठन का नोटिफिकेशन सिस्टम तीन साल में 15 ConcreteCreator सबक्लासेज़ तक बढ़ गया है, और नए इंजीनियर कहते हैं कि क्लास पदानुक्रम में नेविगेट करना मुश्किल है। निदान करें और एक फ़िक्स प्रस्तावित करें। — उत्कृष्ट जवाब: इसे बड़े पैमाने पर पैटर्न की एक वैध लागत के तौर पर पहचानता है (कई छोटी फ़ाइलें, एक नज़र में "सभी सपोर्टेड टाइप्स" देखने की कोई एक जगह नहीं), और मौजूदा Factory Method पदानुक्रम के ऊपर परत की गई एक रजिस्ट्री/इंडेक्स (टाइप-की को creator इंस्टेंसेज़ से मैप करने वाला एक सादा NotificationCreatorRegistry) प्रस्तावित करता है — उस एक्सटेंसिबिलिटी तंत्र को छोड़े बिना खोजने-योग्यता सुधारना जो अब भी अपनी कीमत वसूल रहा है।

1. *Design an ADR recommending team-wide guidance on when to use Simple Factory vs. Factory Method vs. Abstract Factory.* — Excellent answer produces genuine, defensible decision criteria (per Part 3's decision tree) rather than a blanket "always use the most sophisticated option," and explicitly names over-engineering as a real, tracked cost the team should watch for in code review.

हिंदी: एक ADR डिज़ाइन करें जो टीम-व्यापी मार्गदर्शन की सिफ़ारिश करे कि Simple Factory बनाम Factory Method बनाम Abstract Factory कब इस्तेमाल करें। — उत्कृष्ट जवाब एक सामान्य "हमेशा सबसे परिष्कृत विकल्प इस्तेमाल करो" के बजाय

असली, बचाव-योग्य निर्णय मानदंड (Part 3 के डिजीज़न ट्री के अनुसार) पैदा करता है, और ओवर-इंजीनियरिंग को एक असली, ट्रैक की गई लागत के तौर पर खुलकर नाम देता है जिसे टीम को कोड रिव्यू में देखना चाहिए।

1. *A candidate on your team wants to use Factory Method to solve "we need a family of matched UI components." Coach them.* — Excellent answer redirects specifically to Abstract Factory, explains precisely why Factory Method's single-creation-method shape can't cleanly express "must stay consistent as a set," and reviews the coaching angle (helping a colleague reach the right pattern) not just the technical correction.

हिंदी: आपकी टीम का एक उम्मीदवार "हमें मेल खाते UI कंपोनेंट्स का एक परिवार चाहिए" हल करने के लिए Factory Method इस्तेमाल करना चाहता है। उसे कोच करें। — उत्कृष्ट जवाब खास तौर पर Abstract Factory की ओर पुनर्निर्देशित करता है, ठीक-ठीक बताता है कि Factory Method का अकेली-क्रिएशन-मैथड आकार "एक सेट के तौर पर लगातार मेल खाना चाहिए" को साफ़ तौर पर व्यक्त क्यों नहीं कर सकता, और कोचिंग एंगल (एक सहकर्मी को सही पैटर्न तक पहुँचाने में मदद करना) की समीक्षा करता है, सिर्फ़ तकनीकी सुधार नहीं।

1. *How do you evaluate whether a legacy Simple Factory should be left alone or refactored to Factory Method, across a portfolio of a dozen services?* — Excellent answer proposes concrete, checkable signals (branch count and churn rate on the Simple Factory's switch statement; whether per-type *behavior*, not just creation, has started diverging) rather than a subjective "if it feels messy" judgment, making the recommendation auditable and consistent across a large team.

हिंदी: आप एक दर्जन सेवाओं के पोर्टफोलियो में कैसे आकलन करते हैं कि एक विरासती Simple Factory को अकेला छोड़ा जाए या Factory Method में रीफैक्टर किया जाए? — उत्कृष्ट जवाब एक व्यक्तिपरक "अगर यह गड़बड़ लगे" निर्णय के बजाय ठोस, जाँचने-योग्य संकेत प्रस्तावित करता है (Simple Factory की switch स्टेटमेंट पर शाखा गिनती और चर्न दर; क्या प्रति-टाइप व्यवहार, सिर्फ़ क्रिएशन नहीं, अलग होना शुरू हो गया है), सिफ़ारिश को एक बड़ी टीम में ऑडिट-योग्य और सुसंगत बनाते हुए।

1. *Retrospectively, your team applied full Factory Method broadly two years ago as a "best practice," and it's now clear several of those applications were unjustified. How do you address this without blaming the original decisions?* — Excellent answer frames it as normal, healthy requirement drift rather than a mistake, proposes a low-risk simplification pass prioritized by actual maintenance pain rather than a wholesale rewrite, and treats it as a case study for updating the team's own guidance going forward.

हिंदी: पीछे मुड़कर देखें तो, आपकी टीम ने दो साल पहले एक "बेस्ट प्रैक्टिस" के तौर पर व्यापक रूप से पूरी Factory Method लगाई थी, और अब साफ़ है कि उनमें से कई इस्तेमाल अनुचित थे। मूल फ़ैसलों को दोष दिए बिना आप इसे कैसे संबोधित करते हैं? — उत्कृष्ट जवाब इसे एक ग़लती के बजाय सामान्य, स्वस्थ ज़रूरत-बदलाव के तौर पर फ़्रेम करता है, एक संपूर्ण फिर से लिखने के बजाय असली रखरखाव दर्द से प्राथमिकता दिया गया एक कम-जोखिम सरलीकरण पास प्रस्तावित करता है, और इसे आगे के लिए टीम के अपने मार्गदर्शन को अपडेट करने के एक केस स्टडी के तौर पर मानता है।

1. *Contrast Factory Method's role in a monolith versus a microservices architecture where "types" might correspond to entirely different downstream services.* — Excellent answer: in a monolith, Factory Method varies which *class* runs in-process; in a microservices context, the equivalent decision (which downstream service/provider to call) is often better expressed as configuration-driven routing or a strategy/adaptor selected by a registry, since

the "new type" now often means "a new service to integrate with," which is a materially bigger unit of change than "a new subclass."

हिंदी: एक मोनोलिथ बनाम एक माइक्रोसर्विसेज़ आर्किटेक्चर में Factory Method की भूमिका की तुलना करें, जहाँ "टाइप्स" पूरी तरह अलग डाउनस्ट्रीम सेवाओं से मेल खा सकती हैं। — उत्कृष्ट जवाब: एक मोनोलिथ में, Factory Method यह बदलता है कि in-process कौन-सी क्लास चलती है; एक माइक्रोसर्विसेज़ संदर्भ में, समकक्ष फ़ैसला (कौन-सी डाउनस्ट्रीम सेवा/प्रोवाइडर को कॉल करना है) अक्सर कॉन्फ़िगरेशन-प्रेरित रूटिंग या एक रजिस्ट्री से चुनी गई strategy/adapter के तौर पर बेहतर व्यक्त होता है, क्योंकि "नई टाइप" का अब अक्सर मतलब है "एक नई सेवा से इंटीग्रेट करना," जो "एक नई सबक्लास" से भौतिक रूप से बहुत बड़ी बदलाव इकाई है।

1. *When would you actively remove a Factory Method hierarchy and replace it with a single Simple Factory, even though that reads as "less proper" architecturally?* — Excellent answer names the honest, judgment-driven answer: when the codebase's actual growth pattern over time shows the extensibility was never really exercised, and the ongoing readability/navigation cost of the class hierarchy is now larger than the (rarely-collected) benefit — explicitly valuing measured real-world usage over textbook "correctness."

हिंदी: आप कब सक्रिय रूप से एक Factory Method पदानुक्रम हटाएँगे और उसे एक अकेली Simple Factory से बदलेंगे, भले ही यह आर्किटेक्चरल रूप से "कम सही" लगे? — उत्कृष्ट जवाब ईमानदार, निर्णय-प्रेरित जवाब देता है: जब कोडबेस का असली बढ़ोतरी पैटर्न समय के साथ दिखाए कि एक्सटेंसिबिलिटी कभी असल में इस्तेमाल ही नहीं हुई, और क्लास पदानुक्रम की चल रही पठनीयता/नेविगेशन लागत अब (शायद ही कभी वसूल होने वाले) फ़ायदे से बड़ी है — टेक्स्टबुक "सटीकता" के मुकाबले मापे गए असली-दुनिया इस्तेमाल को खुलकर महत्व देते हुए।

Coding problems (solutions in `Factory.php`): 1. Implement a `NotificationCreator` hierarchy (Email/SMS/Push) with a shared `notify()` template method and per-subclass `createNotification()` factory methods, then add a fourth type (WhatsApp) touching zero existing files, with a test proving it. 2. Implement and deliberately break, then fix, the `self`-vs-`static` late-static-binding bug in an inherited factory method, with a before/after identity-check test demonstrating the failure and the fix.

कोडिंग समस्याएँ (हल `Factory.php` में): 1. एक साझा `notify()` टेम्पलेट मेथड और प्रति-सबक्लास `createNotification()` factory मेथड्स के साथ एक `NotificationCreator` पदानुक्रम (Email/SMS/Push) इम्प्लीमेंट करें, फिर शून्य मौजूदा फ़ाइलों को छूते हुए एक चौथी टाइप (WhatsApp) जोड़ें, इसे साबित करने वाले एक टेस्ट के साथ। 2. एक इनहेरिटेड factory मेथड में `self`-बनाम-`static` late-static-binding बग को इम्प्लीमेंट करें और जानबूझकर तोड़ें, फिर ठीक करें, विफलता और फ़िक्स दिखाने वाले पहले/बाद के पहचान-जाँच टेस्ट के साथ।

Total questions delivered: 35 (7 per level × 5 levels), plus 2 coding problems.

कुल दी गई सवालों की संख्या: 35 (7 प्रति स्तर × 5 स्तर), साथ ही 2 कोडिंग समस्याएँ।

▢ APPENDIX

Part 22 — Learning Roadmap & Self-Assessment

Ranked resources: - *Beginner:* the GoF-pattern chapter on Factory Method in any standard design-patterns reference; PHP manual pages on late static binding (`self::` vs. `static::`). - *Intermediate:* Laravel's `Illuminate\Support\Manager` source (directly verified and cited in Part 11) as a real, precisely-scoped example of a Factory-Method-adjacent mechanism. - *Advanced:* comparative reading on Factory Method vs. Abstract Factory vs. Simple Factory across languages with and without classical inheritance (e.g., Go's interface-and-constructor-function idiom from Part 10), to build language-agnostic pattern-recognition rather than memorized PHP/Java syntax.

दर्जाबद्ध संसाधन: - शुरुआती: किसी भी मानक डिज़ाइन-पैटर्न रेफ़रेंस में Factory Method पर GoF-पैटर्न अध्याय; late static binding (`self::` बनाम `static::`) पर PHP मैनुअल पेजेज़। - मध्यवर्ती: Laravel का `Illuminate\Support\Manager` स्रोत (Part 11 में सीधे सत्यापित और उद्धृत) एक Factory-Method-नज़दीकी तंत्र के असली, सटीक रूप से सीमित उदाहरण के तौर पर। - उन्नत: क्लासिकल इनहेरिटेन्स के साथ और बिना वाली भाषाओं में Factory Method बनाम Abstract Factory बनाम Simple Factory पर तुलनात्मक पठन (जैसे, Part 10 का Go का इंटरफ़ेस-और-कंस्ट्रक्टर-फ़ंक्शन तरीका), रटे हुए PHP/Java सिंटैक्स के बजाय भाषा-निरपेक्ष पैटर्न-पहचान बनाने के लिए।

Self-Assessment — MCQs (answer key at the end):

स्वयं-मूल्यांकन — MCQs (अंत में उत्तर कुंजी):

1. What does a `ConcreteCreator` do that a base `Creator` does not? a) Nothing different b) Overrides the factory method to return a specific `ConcreteProduct` c) Implements the `Product` interface directly d) Requires a private constructor

हिंदी: एक `ConcreteCreator` ऐसा क्या करता है जो एक बेस `Creator` नहीं करता? a) कुछ अलग नहीं b) एक खास `ConcreteProduct` लौटाने के लिए factory मेथड को ओवरराइड करता है c) `Product` इंटरफ़ेस सीधे इम्प्लीमेंट करता है d) एक प्राइवेट कंस्ट्रक्टर की माँग करता है

1. What's the key difference between Factory Method and Abstract Factory? a) There is no difference b) Abstract Factory creates a family of related objects; Factory Method creates one c) Factory Method is faster d) Abstract Factory doesn't use interfaces

हिंदी: Factory Method और Abstract Factory में मुख्य अंतर क्या है? a) कोई अंतर नहीं b) Abstract Factory संबंधित ऑब्जेक्ट्स का एक परिवार बनाता है; Factory Method एक बनाता है c) Factory Method तेज़ है d) Abstract Factory इंटरफ़ेस इस्तेमाल नहीं करता

1. In PHP, what does `new static()` do differently from `new self()` inside an inherited method? a) Nothing, they're identical b) `static` resolves to the actual calling subclass at runtime; `self` always resolves to the class where the code is written c) `self` is deprecated d) `static` only works in interfaces

हिंदी: PHP में, एक इनहेरिटेड मेथड के अंदर `new static()`, `new self()` से अलग क्या करता है? a) कुछ नहीं, वे एक जैसे हैं b) `static` रनटाइम पर असली कॉलिंग सबक्लास को रिज़ॉल्व करता है; `self` हमेशा उस क्लास को रिज़ॉल्व करता है जहाँ कोड लिखा गया है c) `self` अप्रचलित (deprecated) है d) `static` सिर्फ़ इंटरफ़ेस में काम करता है

1. Is "Simple Factory" (one method, one switch, no subclassing) a formal GoF design pattern?
a) Yes, it's the simplest form of Factory Method b) No — it's a common, useful idiom, but not one of the 23 GoF patterns c) Yes, it's listed as a structural pattern d) No, it was deprecated in later editions

हिंदी: क्या "Simple Factory" (एक मेथड, एक switch, कोई सबक्लासिंग नहीं) एक औपचारिक GoF डिज़ाइन पैटर्न है? a) हाँ, यह Factory Method का सबसे सरल रूप है b) नहीं — यह एक आम, उपयोगी तरीका है, लेकिन 23 GoF पैटर्न्स में से एक नहीं c) हाँ, इसे स्ट्रक्चरल पैटर्न के तौर पर सूचीबद्ध किया गया है d) नहीं, इसे बाद के संस्करणों में अप्रचलित कर दिया गया था

1. Does Factory Method generally have a meaningful thread-safety/concurrency story? a) Yes, always, like Singleton b) Generally no, since it typically produces fresh, independent objects — except when a ConcreteCreator itself holds mutable shared state c) No, it's impossible to use in multi-threaded code d) Only in Java

हिंदी: क्या Factory Method में आमतौर पर कोई सार्थक थ्रेड-सेफ्टी/concurrency कहानी होती है? a) हाँ, हमेशा, Singleton की तरह b) आमतौर पर नहीं, क्योंकि यह आमतौर पर ताज़ा, स्वतंत्र ऑब्जेक्ट्स बनाता है — सिवाय जब एक ConcreteCreator खुद परिवर्तनशील साझा स्थिति रखता है c) नहीं, मल्टी-थ्रेडेड कोड में इसका इस्तेमाल असंभव है d) सिर्फ़ Java में

1. What SOLID principle is this pattern's strongest, most direct payoff? a) Interface Segregation Principle b) Open/Closed Principle c) Liskov Substitution Principle alone d) None of them

हिंदी: इस पैटर्न का सबसे मज़बूत, सबसे सीधा फ़ायदा कौन-सा SOLID सिद्धांत है? a) Interface Segregation Principle b) Open/Closed Principle c) सिर्फ़ Liskov Substitution Principle d) इनमें से कोई नहीं

1. What is Laravel's `Manager` class family's driver-resolution mechanism closest to? a) Textbook subclass-based Factory Method b) A Simple-Factory-plus-registry hybrid using naming-convention method dispatch c) Abstract Factory d) Singleton

हिंदी: Laravel के `Manager` क्लास परिवार का driver-resolution तंत्र सबसे ज़्यादा किसके करीब है? a) टेक्स्टबुक सबक्लास-आधारित Factory Method b) नामकरण-परंपरा मेथड डिस्पैच इस्तेमाल करने वाला एक Simple-Factory-प्लस-रजिस्ट्री हाइब्रिड c) Abstract Factory d) Singleton

Answer key: 1-b, 2-b, 3-b, 4-b, 5-b, 6-b, 7-b.

उत्तर कुंजी: 1-b, 2-b, 3-b, 4-b, 5-b, 6-b, 7-b.

Scenario questions: 1. A codebase has a `ShapeFactory::create($type)` static method with six branches, used from exactly two call sites, with no history of new shape types being added in three years. A colleague proposes refactoring it to full Factory Method "to follow best practices." Evaluate this proposal. — Expected reasoning: correctly push back, citing the actual usage/growth evidence (Part 20's senior-mistake pattern) rather than deferring to "best practices" as a blanket justification. 2. Your `PaymentMethodCreator` hierarchy (Part 12's ADR scenario) now needs to also produce a matching `RefundHandler` that must use compatible logic

for each payment rail — always as a matched pair. — Expected reasoning: correctly recognize this has become a "family of related objects" requirement and reason through migrating toward Abstract Factory rather than bolting a second, parallel Factory Method hierarchy on awkwardly.

परिदृश्य सवाल: 1. एक कोडबेस में छह शाखाओं वाली एक `ShapeFactory::create($type)` स्टैटिक मेथड है, ठीक दो कॉल-साइट्स से इस्तेमाल होती है, तीन साल में कोई नई शेप टाइप जोड़े जाने का इतिहास नहीं। एक सहकर्मी "बेस्ट प्रैक्टिसेज़ फ़ॉलो करने के लिए" इसे पूरी Factory Method में रीफ़ैक्टर करने का प्रस्ताव रखता है। इस प्रस्ताव का मूल्यांकन करें। — अपेक्षित तर्क "बेस्ट प्रैक्टिसेज़" को एक सामान्य औचित्य मानने के बजाय असली इस्तेमाल/बढ़ोतरी के सबूत (Part 20 का सीनियर-ग़लती पैटर्न) का हवाला देते हुए सही ढंग से पीछे धकेलना। 2. आपका `PaymentMethodCreator` पदानुक्रम (Part 12 का ADR परिदृश्य) अब एक मेल खाता `RefundHandler` भी बनाना है जिसे हर पेमेंट रेल के लिए संगत लॉजिक इस्तेमाल करना चाहिए — हमेशा एक मेल खाती जोड़ी के तौर पर। — अपेक्षित तर्क सही ढंग से पहचानें कि यह अब "संबंधित ऑब्जेक्ट्स के परिवार" की ज़रूरत बन गई है और एक दूसरे, समानांतर Factory Method पदानुक्रम को अजीब तरीक़े से जोड़ने के बजाय Abstract Factory की ओर माइग्रेट करने के बारे में तर्क करें।

One refactoring exercise: Take the Stage 3 (`self`-instead-of-`static`) implementation from Part 19, reproduce the failure with a test, fix it, and add the identity-check test from Part 18 that would have caught it originally.

एक रीफ़ैक्टरिंग अभ्यास: Part 19 से चरण 3 (`self`-केबजाय-`static`) इम्प्लीमेंटेशन लें, एक टेस्ट से विफलता दोहराएँ, इसे ठीक करें, और Part 18 का पहचान-जाँच टेस्ट जोड़ें जो इसे मूल रूप से पकड़ लेता।

One architecture/debugging scenario: A `NotificationCreator` hierarchy has grown to 15 subclasses (Part 21's staff question #1). Produce a short design note: is this hierarchy still earning its keep, what discoverability problem has it introduced, and what would you add (without removing the extensibility mechanism) to fix it.

एक आर्किटेक्चर/डीबगिंग परिदृश्य: एक `NotificationCreator` पदानुक्रम 15 सबक्लासेज़ तक बढ़ गया है (Part 21 का स्टाफ़ सवाल #1)। एक छोटा डिज़ाइन नोट बनाएँ: क्या यह पदानुक्रम अब भी अपनी कीमत वसूल रहा है, इसने कौन-सी खोजने-योग्यता समस्या पेश की है, और (एक्सटेंसिबिलिटी तंत्र हटाए बिना) इसे ठीक करने के लिए आप क्या जोड़ेंगे।

Technical Words Glossary / तकनीकी शब्दों की शब्दावली

English Term	Hindi Translation / हिंदी अनुवाद	Example / उदाहरण
Creational Pattern	क्रिएशनल पैटर्न	Factory Method, Builder, और Singleton तीनों क्रिएशनल पैटर्न हैं।
ConcreteCreator	कॉन्क्रीट क्रिएटर	EmailNotificationCreator , NotificationCreator का एक ConcreteCreator है।
ConcreteProduct	कॉन्क्रीट प्रोडक्ट	EmailNotification , Notification इंटरफ़ेस का एक ConcreteProduct है।
Late Static Binding	लेट स्टैटिक बाइंडिंग	new static() , PHP की late static binding की वजह से सही सबक्लास लौटाता है।
Open/Closed Principle	ओपन/क्लोज़्ड प्रिंसिपल	नई सबक्लास जोड़ना, मौजूदा कोड बदले बिना Open/Closed Principle का पालन करता है।
Dependency Inversion	डिपेंडेंसी इनवर्जन	क्लाइंट कोड कॉन्क्रीट क्लासेज़ के बजाय ऐब्स्ट्रैक्शन्स पर निर्भर करके Dependency Inversion का पालन करता है।
Simple Factory	सिंपल फैक्टरी	एक स्टैटिक मेथड और एक switch , कोई सबक्लासिंग नहीं — यह एक Simple Factory है, GoF पैटर्न नहीं।
Architecture Decision Record (ADR)	आर्किटेक्चर डिज़ीज़न रिकॉर्ड	टीम ने Factory Method क्यों चुना, यह एक ADR में दस्तावेज़ीकृत किया।
Polymorphic Dispatch	पॉलीमॉर्फिक डिस्पैच	factory मेथड कॉल करना, रनटाइम पर सही सबक्लास के ओवरराइड तक पॉलीमॉर्फिक डिस्पैच के ज़रिए पहुँचता है।
Identity Check	पहचान जाँच	get_class() से एक पहचान जाँच self / static बग पकड़ती है।
Behavioral Contract	व्यवहारगत कॉन्ट्रैक्ट	हर ConcreteProduct को Product इंटरफ़ेस का व्यवहारगत कॉन्ट्रैक्ट पूरा करना चाहिए, सिर्फ़ सिग्नेचर नहीं।
Registry	रजिस्ट्री	Laravel का Manager क्लास परिवार एक Simple-Factory-प्लस-रजिस्ट्री हाइब्रिड है।

General Words Glossary / सामान्य शब्दों की शब्दावली

English Word	Hindi Meaning / हिंदी अर्थ	Example / उदाहरण
Unwieldy	बोझिल, संभालने में मुश्किल	"The <code>switch</code> statement had grown unwieldy after two years of additions." दो साल की बढ़ोतरी के बाद <code>switch</code> स्टेटमेंट बोझिल हो गया था।
Blast radius	प्रभाव का दायरा	"A bug in shared code has a much larger blast radius than one in an isolated module." साझा कोड में एक बग का प्रभाव दायरा एक अलग-थलग मॉड्यूल से कहीं बड़ा होता है।
Reflexively	बिना सोचे-समझे	"He reflexively reached for the 'proper' pattern without checking if it was needed." उसने ज़रूरत जाँचे बिना ही "सही" पैटर्न की ओर बिना सोचे-समझे हाथ बढ़ाया।
Underwhelming	कमज़ोर, प्रभावहीन	"The demo felt underwhelming compared to what was promised." प्रदर्शन जो वादा किया गया था उसके मुक़ाबले कमज़ोर लगा।
Auditable	जाँचने-योग्य	"A clear decision rule makes the process auditable across teams." एक साफ़ निर्णय नियम प्रक्रिया को टीमों में जाँचने-योग्य बनाता है।
Preemptively	पहले से, एहतियातन	"She preemptively flagged the risk before anyone asked." किसी के पूछने से पहले ही उसने जोखिम को एहतियातन चिह्नित कर दिया।
Sequencing	क्रमबद्ध करना	"Careful sequencing of the migration avoided any downtime." माइग्रेशन का सावधानीपूर्वक क्रमबद्ध करना किसी भी डाउनटाइम से बचा गया।
Discoverability	खोजने-योग्यता	"Fifteen small files hurt discoverability for new engineers." पंद्रह छोटी फ़ाइलें नए इंजीनियरों के लिए खोजने-योग्यता को नुक़सान पहुँचाती हैं।
Ceremony (figurative)	औपचारिकता, अतिरिक्त बनावट	"The extra classes felt like unnecessary ceremony for such a simple case." इतने सादे मामले के लिए अतिरिक्त क्लासेज़ अनावश्यक औपचारिकता जैसी लगीं।
Drift (noun/verb)	धीरे-धीरे भटकना	"Requirement drift is normal over a two-year period." दो साल की अवधि में ज़रूरत का धीरे-धीरे भटकना सामान्य है।
Earning its keep (idiom)	अपनी कीमत वसूल करना	"Is this abstraction still earning its keep, or is it dead weight now?" क्या यह ऐब्स्ट्रैक्शन अब भी अपनी कीमत वसूल कर रहा है, या अब यह बेकार बोझ है?
Payoff	फ़ायदा, नतीजा	"The payoff only shows up once you actually need a third type." फ़ायदा तभी दिखता है जब आपको सचमुच एक तीसरी टाइप की ज़रूरत पड़े।

Companion file: `Factory.php` — *basic* → *Simple Factory* → *full Factory Method progression*, heavily commented, runnable with `php Factory.php`. Code file is English-only; this handbook is bilingual English + Hindi throughout.

साथी फ़ाइल: `Factory.php` — *basic* → *Simple Factory* → *full Factory Method* क्रम, भारी टिप्पणियों के साथ, `php Factory.php` से रनेबल। कोड फ़ाइल सिर्फ़ अंग्रेज़ी में है; यह हैंडबुक पूरी तरह अंग्रेज़ी + हिंदी द्विभाषी है।